OSTBAYERISCHE
TECHNISCHE HOCHSCHULE
REGENSBURG

RESILENT

# AI-based Anomaly Detection for Electric Vehicle Supply Equipment using LSTM Architecture

An der Fakultät für Informatik und Mathematik der
Ostbayerischen Technischen Hochschule Regensburg
im Studiengang
Wirtschaftsinformatik

eingereichte

## Bachelorarbeit

zur Erlangung des akademischen Grades des
Bachelor of Science (B.Sc.)


**Vorgelegt von:** Yannic Hanel
**Matrikelnummer:** 3296954

**Erstgutachter:** Prof. Dr. Rudolf Hackenberg
**Zweitgutachter:** Prof. Dr. Sebastian Fischer


**Abgabedatum:** 23.08.2025

# Erklärung zur Bachelorarbeit

1. Mir ist bekannt, dass dieses Exemplar der Abschlussarbeit als Prüfungsleistung in das Eigentum der Ostbayerischen Technischen Hochschule Regensburg übergeht.
2. Ich erkläre hiermit, dass ich diese Abschlussarbeit selbständig verfasst, noch nicht anderweitig für Prüfungszwecke vorgelegt, keine anderen als die angegebenen Quellen und Hilfsmittel benutzt sowie wörtliche und sinngemäße Zitate als solche gekennzeichnet habe.

Regensburg, den July 28, 2025

---

Yannic Hanel

# Abstract

AI is important ...

# Contents

# List of Figures

# List of Tables



# I List of Abbreviations

Electric Vehicle Supply Equipment (EVSE)
Electric Vehicles (EVs)
Charging Stations (CS)
Artificial Intelligence (AI)
Isolation Forest (iForest)
Recurrent Neural Network (RNN)
Exploratory Data Analysis (EDA)
Deep Neural Network (DNN)
Application Programming Interface(s)(APIs)
Information Technology (IT)
Denial-of-Service (DoS)
Principal Component Analysis (PCA)

# 1 Introduction

## 1.1 Motivation and Relevance of Anomaly Detection in Electric Vehicle Supply Equipment (EVSE)

Electric Vehicle Supply Equipment (EVSE) has gained significant importance in recent years [EVSE24]. Analyzing the market sales of electric vehicles (EVs), we observe a remarkable growth: in 2012, only 120,000 EVs were sold, whereas by 2021, sales had surged to nearly 6.6 million. And to achieve net-zero CO2 emissions by 2050, as planed by the European Union [EU2050], the number of EVs must continue to rise substantially—requiring an 11-fold increase from current levels by 2030 alone [IEA22].
To manage the much higher demand for EVs on our roads, we can assume that the market for EVSE will grow similarly to the resale of EVs. The correlation between the number of EVs and the number of EVSE is high. If we analyze the reasale from the last few years, we can see that the top five countries in the European Union like Germany, Netherlands, France, and Belgium with the best EVSE infrastructure are also the top five with the largest market share of EVs [ACEA23].

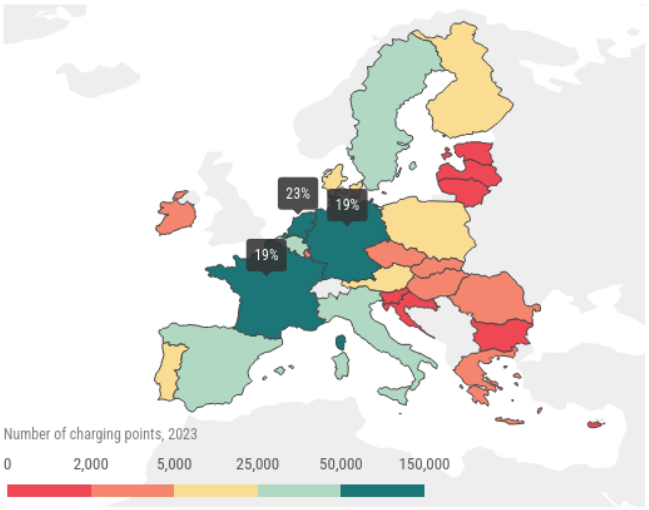


Figure 1: Availability of public charging points and market share of electric cars in the EU[ACEA23].

With the growing number of EVSE, particularly Charging Stations (CS), the interest of cybercriminal groups in attacking these systems is also increasing. At first glance, EVSE systems may not appear to be direct targets for accessing critical data or manipulatißon. However, they can serve as entry points to gain access to more sensitive or private information [EntryPoints25]. The following figure illustrates how EVSE components, such as charging stations, can act as access nodes to critical or private data.

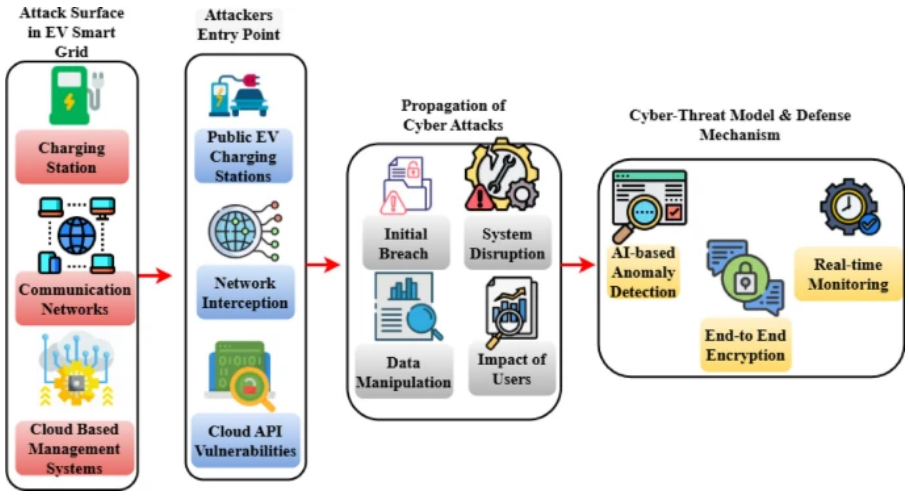


Figure 2: EVSE as an access node to critical data[EntryPoints25].

With the access to these kind of Data the critical attacks like data manipulations, data theft, distrub of service up to complete system failure are possible[EntryPoints25]. That does not only comes with a high financial risk, it also affect the trust of the users in the EVSE systems and potentially slowing down the adoption of EVs in general.
If we provide end users with a stable and secure EVSE infrastructure, consumer confidence in switching to electric vehicles will be significantly higher. This, in turn, ensures a consistent growth in the adoption of both EVs and EVSE systems[EVSE-Cyber25].
As seen in this first subchapter, the rapid growth of EVs and EVSE infrastructure also requires a robust monitoring and anomaly detection system to ensure the security of the EVSE infrastructure. Only if we can ensure the security of the EVSE infrastructure can we maintain user trust in the energy transition and avoid major cybersecurity headlines in the future.

Ensuring high security standards for EVSE systems is of critical importance, yet difficult to achieve. The EVSE infrastructure constitutes a complex system that includes not only charging stations but also sophisticated backend systems and communication protocols. These components and software solutions are typically developed, published, and maintained by various vendors, which makes securing the entire system a challenging task.
Combining all the different software from vendors into a secure infrastructure is challenging enough, but the EVSE infrastructure, especially charging stations (CS), is also regularly installed in public places. Even when not in a public setting, these installations are often outdoors, where security measures are significantly lower than in buildings protected by walls, doors, and other physical barriers.
In this paper, we focus on wallboxes, which are generally smaller than CS and are typically used in private settings. They are often installed directly on house walls or in garages, and thus still form part of the overall EVSE infrastructure.
The fact that they are installed in private areas initially suggests a higher level of security.

However, since these areas are still outdoors, we must assume that there are no additional physical security measures beyond the wallbox itself. Therefore, it is crucial to develop a system that not only protects against software-based attacks, such as Denial-of-Service (DoS) or Distributed Denial-of-Service (DDoS), but also considers other potential attack vectors such as firmware manipulation and unauthorized data interception. In addition, the system must secure the hardware against physical threats like tampering, theft, or other intrusions.
In our project, we use wallboxes from the [ReSiLENT] project, provided by [StoehrMobility]. The software is developed by [ChargeIQ] and already includes basic security measures to protect against external threats. This project is also carried out in cooperation with [ASVIN]. In the following section, we will take a closer look at the internal components of the wallbox.

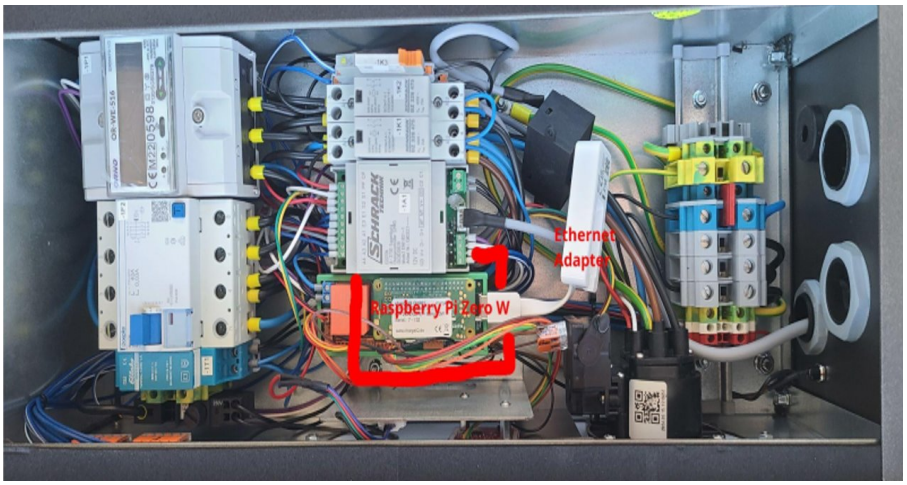


Figure 3: Interior view of the wallbox[Han25].

In this figure, the internal components of the wallbox are visible. The wallbox is equipped with a Raspberry Pi Zero W, which serves as the main controller for the planned peer-to-peer communication between the wallboxes. This device is also intended to host of the detection system developed as part of this project.
Located behind the Raspberry Pi Zero W is another Raspberry Pi, which is responsible for communication with the backend, including the transmission of user data and handling of payment processes from the Charging process. This component is secured and programmed by *ChargeIQ GmbH*[ChargeIQ].
The sensitive hardware components are enclosed in an aluminum casing designed by *Stöhr Mobility GmbH*[StoehrMobility], which provides a degree of protection against physical attacks. However, upon closer inspection of these physically secured components, it must be acknowledged that the protective measures can be relatively easily bypassed. This has already been observed and documented by our colleague Lea Laux in her work [Laux24].
As already mentioned, the wallbox also processes personal data of users, such as their user ID and payment information, in communication with the backend server. This requires special

attention in the context of the DSGVO / GDPR, as additional requirements for the protection of personal data like data minimization, storage limitation [GDPR5] and legal obligation [GDPR6] must be taken into account throughout the entire EVSE infrastructure.
Security requires significant computational resources, particularly in the domain of anomaly detection. As shown in Figure 3, the wallbox is equipped with a Raspberry Pi Zero W, which offers relatively sufficient resources for a single-board computer. However, when processing and analyzing over 1,000 hardware events to detect potential anomalies, the system quickly reaches the limits of its hardware capabilities this is just a small example of the challenges we face in securing EVSE infrastructure.
Due to the combination of digital and physical threats, the involvement of multiple vendors, and strict data protection requirements, securing Electric Vehicle Supply Equipment (EVSE) infrastructure is a complex task and brings a lot of Challenges.

## 1.2 Overview of AI-based approaches

Artificial intelligence is often used in different contexts, therefore we have to define what we mean by AI-based approchaes and what different approaches exist. At first it is a leading technology in the current digital transformation and is crucial to move forward the fourth industrial revolution[AI-Definition].
The Goal of AI is to create machines that can perform tasks that typically require human intelligence, such as understanding natural language and regonize patterns in data. Exspecially detection of patterns is in this work from Relevance. We can devide the different AI-based approches into two sub categories:

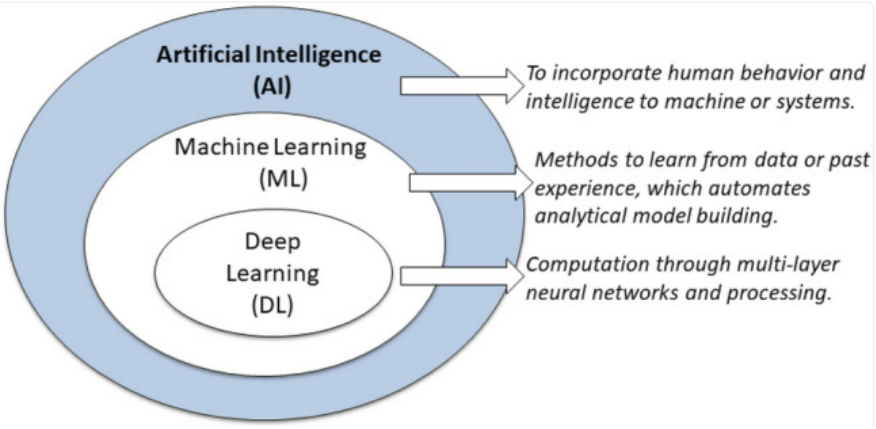


Figure 4: An illustration of the position of machine learning (ML) and deep Learning (DL) within the area of artificial intelligence (AI)[AI-Definition]

these can be further divided into three categories:

- **Supervised Learning:** This approach involves learning from labeled data to identify patterns and make predictions. It is commonly used in tasks such as email spam classification or price prediction, where the algorithm is trained on known input-output pairs.

- **Unsupervised Learning:** This data-driven method is used to uncover hidden patterns in unlabeled data. Techniques such as clustering and dimensionality reduction are applied to identify structures within the data, for example, to segment customer groups or discover business models.
- **Semi-Supervised Learning:** This approach combines aspects of both supervised and unsupervised learning by utilizing a small amount of labeled data alongside a larger pool of unlabeled data. It is particularly useful when labeling data is expensive or time-consuming, but large amounts of unlabeled data are available.

[AI-Definition]

In the following table, we categorize the different approaches alongside their corresponding machine learning architectures. This allows us to identify the advantages and disadvantages of each method and to gain a clearer understanding of which approach is most suitable for producing actionable results in real-world deployments.

| **Approach** | **Examples** | **Advantages** | **Disadvantages** |
|---|---|---|---|
| Supervised Learning | Decision Trees, Random Forest, LSTM | High prediction accuracy, Clear objectives | Requires a lot of labeled data, Risk of overfitting |
| Unsupervised Learning | K-Means, Isolation Forest | Finds patterns without labels, Useful for unlabeled data | Hard to interpret, No clear target |
| Semi-Supervised Learning | Self-training | Improves performance with few labeled data | Complex to implement |

Table 1: Comparison of machine learning approaches with examples, advantages, and disadvantages. Own illustration based on [AI-Definition].

After gaining an overview of the different machine learning approaches, we can now delve deeper into the field by examining various methods for anomaly detection and identifying the most effective models for detecting typical anomalies.

## 1.3 Research Questions

The following research questions guide this thesis on **AI-based Anomaly Detection for Electric Vehicle Supply Equipment using LSTM Architecture**:

- What types and characteristics of anomalies occur in electric vehicle supply equipment (EVSE)?

- How can relevant features be identified and selected to effectively train machine and deep learning models for anomaly detection?
- How does the performance of LSTM-based models compare to that of traditional or alternative AI-based methods in terms of detection accuracy and false positives?
- What practical challenges arise when implementing such anomaly detection systems in real-world charging infrastructure?

These research questions address both the technical and real-world aspects of developing and applying anomaly detection, providing the analytical framework for the subsequent chapters of this thesis.

## 1.4 Structure of the Thesis

This thesis is divided into ten chapters, each building upon the previous to explore the use of AI-based methods—specifically LSTM architectures—for anomaly detection in Electric Vehicle Supply Equipment (EVSE).
Chapter 1 introduces the motivation and relevance of the topic, outlines the role of anomaly detection in the context of EV infrastructure, and formulates the central research questions. Additionally, it provides an overview of prior AI-based approaches and concludes with this structural outline. Chapter 2 summarizes the key theoretical foundations necessary for the thesis, including the basics of anomaly detection, a short introduction to tree-based models such as decision trees and random forests, and a focused explanation of the LSTM architecture.
Chapter 3 presents related work, covering research on security aspects and anomaly detection mechanisms specific to EVSE, as well as a broader review of AI-based detection methods applied in similar domains. Chapter 4 describes the dataset creation and acquisition process, detailing both public datasets and custom setups involving Prometheus monitoring tools. Chapter 5 follows with an exploratory data analysis (EDA), intended to uncover key patterns and differences between the public and self-generated datasets.
In Chapter 6, the process of data preprocessing and feature selection is discussed, including the identification of relevant features and the use of feature importance methods such as decision trees. Chapter 7 focuses on the LSTM model itself, describing its architecture, the training process, and the chosen strategies for hyperparameter tuning.
The results of the approach are then evaluated in Chapter 8, which outlines the experimental setup, evaluation metrics, and key results of the applied anomaly detection system. Chapter 9 provides a detailed discussion of the findings in relation to the research questions, highlighting limitations and potential practical applications. Finally, Chapter 10 concludes the thesis with a summary of the work and a reflection on future directions.
This structure ensures a logical and methodical development of the topic, from theoretical foundations to implementation, evaluation, and critical interpretation.

# 2 Fundamentals and Key Terminologies

## 2.1 Basics of Anomaly Detection

In the previous section, we took a closer look at what Artificial Intelligence is and how useful it can be in the field of EVSEs. In this section, we delve deeper into the area of Anomaly Detection and its fundamental concepts.

Anomaly Detection is the process of analyzing data patterns to identify outliers — in other words, data points that deviate significantly from the norm. This technique can be applied in a wide range of domains, including credit card fraud detection, insurance and healthcare monitoring, intrusion detection in cyber-security, fault detection in safety-critical systems, military surveillance, and also in the field of EVSEs.[BAN09]

Outliers are data points that differ significantly from the majority of the data. In the paper by Hodge and Austin, they define outliers as "data points that are inconsistent with the rest of the data." The paper also includes some informative graphics that illustrate the different types of outliers and how they can be detected.[HA04]

Type 1 is the most common form of outlier, where data points are generally clustered around

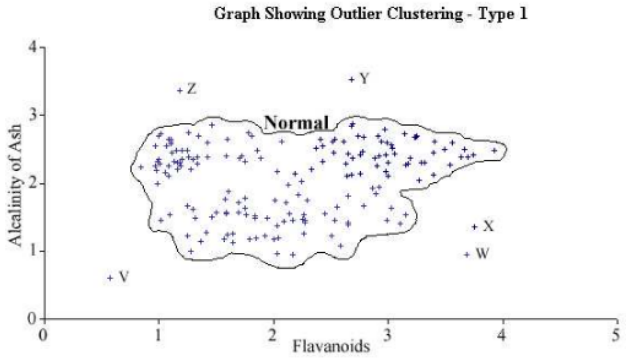


Figure 5: Data distribution classified by Type 1 – Outlier Clustering. The data is adapted from the Wine dataset (Blake and Merz, 1998)[HA04].

a central value, but a few points differ significantly from the rest. This can be seen in the figure above, where most data is centered, yet a few outliers deviate clearly.

Such outliers can be detected using a simple z-score algorithm, where the mean $\mu$ and the standard deviation $\sigma$ of the data are calculated. The z-score is then computed using the formula: $Z = \frac{X-\mu}{\sigma}$. Estimated z-scores greater than 3 or less than -3 can be considered anomalies. These anomalies should be given increased attention during data analysis. Additionally, anomalies can be contextualized by parameters such as time or location to improve their identification.[Per19]

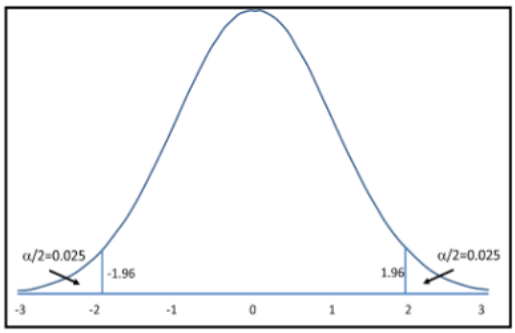


Figure 6: Z-score with critical region[Per19]

But befor you can calculated the z-sorce you need an extensive data collection and preprocessing. This is also the first step in the Anomaly Detection process. The data must be cleaned, normalized, and transformed to ensure that it is suitable for analysis. This may involve removing duplicates, handling missing values, and scaling the data to a common range.
After the data is preprocessed, feature engineering is performed to extract relevant features from the data. Feature engineering involves selecting and transforming the data this can sometimes also to create new features that can improve the performance of the anomaly detection model. Also data reduction is a relevant step in this process, where the data is reduced to a smaller set of features that are most relevant for the anomaly detection task.[Mah22]
If you are done with the date collection and preprocessing, you have to select a suitable model for your case of anomaly detection. But befor we analyse the different models, have take a look at the typical challanges of anomaly detection.
Like mentioned befor to detect anomalys we have to analyze data patterns to identify outliers. But to determine the correct border between normal and abnormal data can be challanging and depends exprem of the context of the data.
The EVSE infrasturcute is change rapidly, not just with new technologies the charging stations change with every new charing process and comminication with other CS in the network. This makes it difficult to define a static outliner theresold. Instead its expremly important to adapt the model to the current state of the EVSE network and the charging process to gernerate dynamic detection in all situations in the dynamic EVSE environment.[BAN09]
Before the z-score can be calculated, extensive data collection and preprocessing are required. This forms the first step in the Anomaly Detection process. The data must be cleaned, normalized, and transformed to ensure it is suitable for analysis. This may involve removing duplicates, handling missing values, and scaling the data to a common range.
After preprocessing, feature engineering is performed to extract the most relevant characteristics from the dataset. Feature engineering involves selecting and transforming the data — and sometimes even creating new features — to improve the performance of the anomaly detection model. Additionally, data reduction is a crucial part of this process. It focuses on reducing the dataset to a smaller subset of features that are most relevant to the anomaly detection task.[Mah22] his step is of high relevance for the dataset used in this work, with further details

provided in the *Data Acquisition and Preprocessing* chapter.
Once data collection and preprocessing are completed, the next step is to select a suitable model for the specific anomaly detection use case. But before we analyze the different models, it is important to examine the typical challenges associated with anomaly detection.
As previously mentioned, detecting anomalies requires analyzing data patterns to identify outliers. But determining the line between normal and abnormal behavior is challenging and highly dependent on context.
The EVSE infrastructure is also evolving rapidly — not only due to new technologies, but also because charging stations behave differently during each new charging process and through communication with other charging stations in the network. This variability makes it difficult to define a static outlier threshold. Therefore, it is essential to adapt the anomaly detection model to the current state of the EVSE network and the ongoing charging processes. This enables dynamic detection capabilities that remain effective under all conditions in a highly dynamic EVSE environment.[BAN09]

## 2.2 Introduction to Decision Trees, Random Forest, Isolation Forest and LSTM

In this section, we will provice a brief introduction to serveral machine learning models who are categorized as supervised learningn models. These models are also the most relevant for our project.
The **Decision tree** classifiers are the most well known and one of the powerful models in various fileds. It works by creating a tree-like strucutere with Nodes and Branches and every Node is an Attribute or feature and the Branch can referent the characteristic of the Attribute[DecisionTreeExplain24].

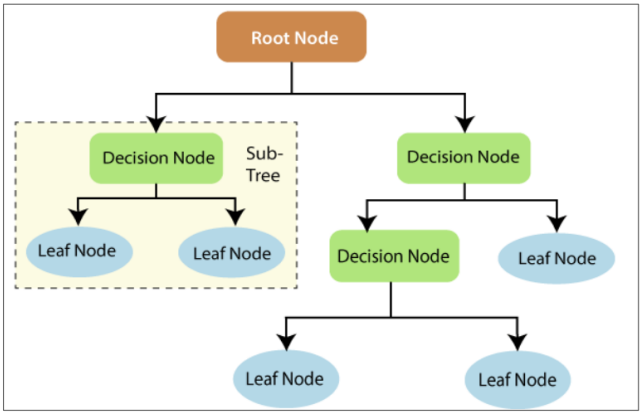


Figure 7: Decision Tree[DecisionTreeExplain24]

The decision rule is not a black box and can be easily interpreted. It compares the value of the feature with a threshold and then decides which branch to follow. It is not only used in

Machine Learning, but is also common in fields like image processing and pattern recognition, due to its simple analysis and high accuracy. Despite these advantages, it has a high risk of overfitting, especially with small datasets.[DecisionTreeExplain24]
The **Random Forest** is an ensemble method that combines multiple decision trees that work independently of each other. Each decision tree is created using a random vector to improve variance and reduce overfitting.
The final decision is made by majority vote of all the trees together. This method is more robust against overfitting and can handle large datasets with high dimensionality. Two key vectors influence the performance of the Random Forest: the correlation between the trees and the strength of the individual trees.[Bre01]
The **Isolation Forest** is an innovative approach for anomaly detection. While conventional methods typically create a model of "normal" behavior and classify deviations from it as anomalies, the iForest follows the principle of directly isolating anomalies.[LTZ08]
The explanation for this is that anomalies are characterized by their rarity and by being significantly different in their attributes. Therefore, they are easier to separate from the majority. This model brings advantages such as robustness with regard to dimensionality and irrelevant features. On the other hand, *swamping* is one of the disadvantages, where normal instances that are too close to anomalies can prevent anomalies from being isolated and thus make them harder to detect.[LTZ08]
A **Long Short-Term Memory (LSTM)** is a special type of Recurrent Neural Network (RNN) designed to model sequential data and long-term dependencies. Especially the long-term dependencies have high priority in this project, where the focus is on detecting anomalies over longer time periods. LSTMs use memory cells and gating mechanisms to store information over long time steps reliably. Despite their many advantages, they come with many different parameters, making them difficult to tune. Additionally, they represent a black-box model, meaning it is hard to understand why the model made a certain decision.[SM19]

| **Machine Learning Model** | **Advantages** | **Disadvantages** |
|---|---|---|
| Decision Tree | Easy to understand and interpret | High possibility of overfitting |
| Random Forest | Less overfitting than a single decision tree; handles high-dimensional data | Not well-suited for small datasets |
| Isolation Forest | Robust to high-dimensional and irrelevant features | Swamping effect in presence of nearby normal instances |
| Long Short-Term Memory | Effective at modeling long-term dependencies | Black-box nature and complex parameter tuning |

Table 2: Comparison of supervised learning models with advantages and disadvantages. Own illustration based on [DecisionTreeExplain24][Bre01][LTZ08][SM19].

## 2.3 Overview of LSTM (Long Short-Term Memory) Architecture

testets

In this chapter, I want to provide a detailed overview of the LSTM architecture, as it is the most frequently used and complex model in my work.
Long Short-Term Memory, or LSTM for short, was developed in 1997 by Sepp Hochreiter and Jürgen Schmidhuber as a solution to the vanishing gradient problem that occurs in traditional Recurrent Neural Networks (RNNs) [SM19]. It allows the network to learn dependencies over much longer time sequences, which is particularly important in my datasets, where attacks can be insidious and not immediately obvious.
This addresses one of the core limitations of traditional RNNs, which struggle to learn long-term dependencies due to the vanishing gradient problem. When training these networks, the error signals either explode—causing oscillating weights and unstable learning—or vanish, preventing the network from learning dependencies over extended time periods. Standard RNNs can typically bridge about 5 to 10 time steps, whereas LSTMs are capable of learning dependencies over 1,000 or more time steps [SM19], LSTMs process inputs sequentially, with a constant runtime of $\mathcal{O}(1)$ per time step.
But how does this work, and how is it possible to achieve this with constant time complexity? The LSTM architecture consists of six different internal states that are processed in a specific sequence to achieve this goal.

- **Forget Gate:** The forget gate calculates a dynamic value between 0 and 1 that scales the previous cell state. A value of 0 means "forget everything" and a value of 1 means "keep everything". This allows the network to selectively forget information that is no longer relevant.
- **Input Gate:** The Input gate decide how much of the new information should be added to the new cell state at the current time step. It is a kind of protective mechanism that tries to prevent irrelvant inputs from being added to the cell state.
- **Candidate Values:** The candidate values are the new information that get in form of squased values that get written into the memory cell.
- **Cell State Update:** The cell state is updated by simultaneously scaling down the existing state via the forget gate and adding in the gated candidate values. This allows the network to maintain a stable memory while also incorporating new information.
- **Output Gate:** The output gate determines which parts of the cell state should be realy becomes the output of the LSTM and protecting in this way the network from sharing unnecessary information too early.
- **Hidden State:** The hidden state is the final output of the LSTM for the current time step and what the outpute gate allows to be shared with the next time step.

[SM19]

| **Machine Learning Model** | **Advantages** | **Disadvantages** |
|---|---|---|
| **Long Short-Term Memory** | Excellent long-term memory capabilities | Fixed network topology – cannot dynamically adjust memory capacity |
| | Selective information processing through gates | Complexity – more parameters than simple RNNs |
| | Stable learning without gradient explosion/vanishing | Modularization challenges – unclear how to optimally structure networks |

Table 3: Strengths and Limitations of LSTM. Own illustration based on [SM19].

# 3 Related Work

## 3.1 Recent Advances in Cybersecurity and Anomaly Detection for EVSE

In the emerging field of EVSE, numerous studies focus either on electric vehicle supply equipment itself or on anomaly detection methods. However, the specific combination of anomaly detection applied to EVSE is not yet widely explored. Therefore, the paper [EDBF24], titled *Enhancing EV Charging Station Security Using a Multi-dimensional Dataset*, is highly relevant to this research. The authors present a dataset used for anomaly detection that includes similar features such as hardware events, power consumption, and other HCP-related data. The paper focuses on various attack scenarios and attempts to detect them using several machine learning models.

Another relevant contribution in this field is the paper [EVSE24], titled *Regression Based Anomaly Detection in Electric Vehicle State of Charge Fluctuations Through Analysis of EVCI Data* which not only highlights the importance of anomaly detection in EVSE, but also provides a comprehensive overview of a realistic charging station scenario. The setup includes four different charging boards, six charging ports, and a battery energy storage system (BESS) handling various types of EVs. The study monitors the state-of-charge (SoC) values of EVs, compares them with expected values, and detects outliers that may indicate anomalies. These detections are performed using machine learning methods within a dynamic environment.

## 3.2 Performance Comparison of Deep Learning Approaches for EVSE Anomaly Detection

In the following, it is important to summarize which machine learning and deep learning algorithms have shown promising performance in the field of anomaly detection for EVSE. Since the data involves time series and hardware-related measurements, the paper [Ali20], titled *Deep Learning-based Intrusion Detection System for Electric Vehicle Charging Station*, provides a valuable overview of suitable deep learning models in this context.
The authors compare a Deep Neural Network (DNN) with a Long Short-Term Memory (LSTM) model and conclude that the LSTM architecture achieves high accuracy—over 99%—after only a few training epochs. This supports the later decision in this thesis to select the LSTM model for better performance and anomaly detection in EVSE environments.

# 4 Data Acquisition

## 4.1 Data Sources for EVSE Anomaly Detection

In line with the approach described in the paper [EDBF24] *Enhancing EV Charging Station Security Using A Multi-dimensional Dataset : CICEVSE2024*, I recorded Hardware Counter Performance (HCP) kernel events and tracepoint events to classify anomalies in Electric Vehicle Supply Equipment (EVSE) data. For efficient monitoring, I also utilized the `perf` tool, a robust Linux performance analysis utility capable of capturing a variety of performance data, including hardware events, software events, and tracepoints [con25]. This tool offers flexible and effective monitoring of system performance and is instrumental in diagnosing issues efficiently.

To perform the data acquisition on a Raspberry Pi Zero W, I used the `perf_exporter`[Hod19] to record up to 900 Tracepoint-events, which were selected based on the CICEVSE2024 dataset. Initially, I developed a Python script to capture kernel and HCP events. However, when expanding to about 110 events, my developed script's runtime increased to approximately 9 seconds, which proved insufficient for real-time analysis. Consequently, I investigated the internal workings of the `perf` tool to understand how it manages specialized events and optimizes recording.

As detailed in *Linux perf event Features and Overhead*[Wea15], `perf` supports multiplexing, allowing the measurement of more events than there are available hardware counters—albeit at the cost of accuracy loss and increased system overhead. Recent updates can result in resource usage exceeding 20%, compared to approximately 2% in older tool versions. To minimize these inaccuracies, I grouped events per CPU, exploiting up to 8 available counter registers and limiting each group to a maximum of 6 events.

For the monitoring interval, I followed recommendations from the literature[Lim+14], which addresses multiplexing and accuracy trade-offs. The suggested minimum monitoring interval is around 10 ms for reliable results. Balancing the number of counter registers and striving for good measurement accuracy, I configured groups of 6 hardware events per group, with a monitoring interval of 0.1 seconds. This configuration yielded data with sufficient accuracy to detect anomalies in the EVSE dataset.

To further optimize efficiency and accuracy, I transitioned to developing the data acquisition script in Go. The Go programming language offers low-level hardware access and, with available libraries, integrates more seamlessly with the `perf` tool compared to Python.
Additionally, the script exports all monitored hardware events from `perf` into a Prometheus database, enabling centralized and convenient access to all collected data. This setup streamlines subsequent analysis, visualization, and anomaly detection activities.
The script is provided in the following section.

Here you can find two functions of the script the first one is used to format the data for Prometheus and the second one is used to monitor the data with the `perf` tool. The whole script you can find in the appendix 10.

```
// formatForPrometheus converts the filtered performance data into Prometheus-compatible metrics format.
func formatForPrometheus(data string) string {
    lines := strings.Split(data, "\n")
    var result []string
    for _, line := range lines {
        parts := strings.Fields(line)
        if len(parts) >= 2 {
            // Clean up the metric name
            metricName := strings.ReplaceAll(parts[1], ".", "_")
            // metricName = strings.ReplaceAll(metricName, "-", "_")
            // Remove decimal points from the number and parse it
            metricValue := strings.ReplaceAll(parts[0], ".", "")
            if value, err := strconv.ParseFloat(metricValue, 64); err == nil {
                // Format value appropriately for Prometheus (integer or float)
                if value == float64(int(value)) {
                    result = append(result, fmt.Sprintf("%s %d", metricName, int(value)))
                } else {
                    result = append(result, fmt.Sprintf("%s %f", metricName, value))
                }
            }
        }
    }
    return strings.Join(result, "\n")
}

// getPerfData runs `perf stat` with the specified events for a short duration (0.1s)
// and returns the raw output as a string.
func getPerfData(events []string) (string, error) {
    eventList := strings.Join(events, ",")
    cmd := exec.Command("perf", "stat", "-e", eventList, "--", "sleep", "0.1")
    output, err := cmd.CombinedOutput()
    if err != nil {
        return "", err
    }
    return string(output), nil
}
```

Listing 1: Go script for performance monitoring via perf for Prometheus

## 4.2 Integration of Prometheus

In the research on which database would be suitable for centralized data storage and analysis, `Prometheus` was a key focus. Not only is it an open-source project [20125], but it is also well known in the industry as a reliable monitoring solution for complex systems. It has a large community, along with comprehensive documentation and strong support, which are essential for maintaining a stable IT environment.
Another reason for choosing `Prometheus` is its seamless integration with Grafana for visualization, as well as its powerful PromQL query language. Additionally, `Prometheus` can store metrics in a time series database, which is particularly important for our detection systems [BMM24].
`Prometheus` consists of several components that are required for its operation. The core of the system is the `Prometheus` server, which collects and stores metrics and provides access to them through the PromQL query language.
Client libraries are directly integrated into the application code and expose a client-server interface. The `Prometheus` server periodically fetches data from them — by default every 15 seconds, although this interval is configurable.
In addition to the core components, `Prometheus` also supports optional components:

- **Push Gateway**: For external or short-lived metrics
- **Alertmanager**: For alerting and notifications
- **Exporters**: For monitoring additional systems, databases, or APIs
- **Visualization tools**: Such as Grafana

[SM24]

As explained in the previous subsection, `Prometheus` collects data via metrics exporters (see code snippet 16). After the data is collected and stored centralized in the `Prometheus` time series database, we developed a Python script that allows exporting the data within a defined time range and with specified time steps.
This export functionality is essential for enabling further analysis, including exploratory data analysis, using external tools such as Python or statistical software. The following code snippet illustrates the script used to export `Prometheus` data in CSV format:

```
1
2 //TODO update the skript ohne cpu aufteilung
3
4 PROMETHEUS_URL = "http://192.168.1.25:9090"
5 STEP = "10"  # seconds
```

```
CHUNK_HOURS = 2

start_time = datetime(2025, 7, 18, 13, 36, 17, 210000)   # 210000 = 210ms
end_time = datetime(2025, 7, 18, 16, 0, 7, 210000)

def main():
    chunks = build_chunks(start_time, end_time, CHUNK_HOURS)
    data = defaultdict(dict)
    timestamps = set()
    values_dict = {}
    # Mapping: (Feature, cpu_label) -> passender Querystring
    metric_query_map = {}

    for feat in FEATURES:
        prom_name = feat.replace("-", "_")
        url = f"{PROMETHEUS_URL}/api/v1/query"
        hit = False

        # 1. prom_name (ohne CPU)
        params = {'query': prom_name}
        try:
            resp = requests.get(url, params=params)
            resp.raise_for_status()
            results = resp.json()['data']['result']
            if results:

                for series in results:
                    cpu_label = series['metric'].get('cpu', 'no_cpu_label
                        ')
                    value = float(series['value'][1])
                    values_dict[(feat, cpu_label)] = value

                    metric_query_map[(feat, cpu_label)] = prom_name
                hit = True
        except Exception:
            pass
```

Listing 2: Python script for exporting data from `Prometheus` in CSV format

Here you can see a short summarization of the key advantages of `Prometheus`, which make it a suitable choice for our data acquisition and live monitoring needs:

| **Advantage** | **Description** |
|---|---|
| Powerful PromQL query language | Enables complex and efficient queries on metric data. |
| Centralized data storage | Stores all metrics centrally as time series data. |
| Extensive documentation and community support | A large community and comprehensive documentation ensure quick support and continuous development. |
| Seamless integration with Grafana dashboards | Provides easy integration with Grafana for visual dashboards and monitoring. |
| Open source and free | Free and open-source, offering broad availability and high customizability. |

Table 4: Key advantages of `Prometheus`.
[BMM24][SM24]

# 5 Exploratory Data Analysis

## 5.1 Exploratory Data Analysis in the Context of EVSE Anomaly Detection

So far, we have provided an overview of the relevance and challenges involved in securing EVSE environments. We discussed the fundamental knowledge required before developing an anomaly detection system, gained insight into related work in this area, and explored how it can inform our own approach.
We have also examined how data can be acquired from EVSE environments and how `Prometheus` can be used to collect and store this data.
All of these sections and subsections are essential for understanding the context of our work and for showing how it is possible to develop an anomaly detection system for EVSE environments based on the analysis of hardware events and the collected data.
In this section, we focus on the Exploratory Data Analysis (EDA) of the collected data. EDA is a crucial step in understanding the data, identifying patterns, and preparing for further analysis or modeling. We use Python and its data analysis libraries within a Jupyter Notebook environment to visualize and explore the collected data.

This chapter is of particular importance because it defines the quality of the results in the following chapters and significantly influences how well anomaly detection using machine learning models will perform and how quickly they will learn. Therefore, we need to answer the following questions:

- Can we observe meaningful distributions in the features over time and in relation to attacks?
- Can we identify and define the most important features?
- Is there any noise in the data, and how significant is it?
- How strong are the correlations between the different features?

Answering these questions through Exploratory Data Analysis (EDA) provides a foundation for effective feature selection and model performance [Goo].

To convert the data into more useful formats, I used **Pandas**, a fast, powerful, and easy-to-use open-source data analysis tool for Python. It provides expressive data structures such as DataFrame and Series for working with structured and time series data.
I primarily used Pandas to convert my data, which was originally stored in `.csv` files, into `.parquet` files. Additionally, I used it to manipulate the dataframes by dropping cells, adding information such as "Attack" or "Benign" tags to almost 7000 rows. This preprocessing step was essential to clean the data before applying machine learning algorithms.
Some other useful key features of Pandas include:

- Tools for reading and writing data between various formats
- Flexible reshaping and merging of data
- Powerful grouping, aggregation, and transformation operations

[The25]
To create diagrams that help me better understand the data, I used a popular plotting library in Python called **Matplotlib**. Specifically, I used the `matplotlib.pyplot` module, which is a state-based interface to the Matplotlib library and provides a MATLAB-like way of creating static, animated, and interactive visualizations in Python.
Matplotlib is well-suited for quickly prototyping visualizations thanks to its simple and readable syntax. This allowed me to create a wide range of plots, including:

- **Line Plots:** Display data points connected by straight lines, useful for visualizing trends over time [Gee22d].
- **Scatter Plots:** Present individual data points as dots in a two-dimensional space, commonly used to show correlations between variables [Gee22e].
- **Bar Charts:** Use rectangular bars with variable lengths to represent values, making it easy to compare quantities across different categories [Gee22a].
- **Histograms:** Divide continuous data into bins and show the frequency or count of data points within each bin [Gee22c].
- **Heatmaps:** Visualize the relationship between multiple variables in a matrix format using color gradients [Gee22b].

These visualizations enabled me to better understand the data structure, identify patterns, and prepare the dataset for further processing.
I also used the Seaborn libary it is like the other 2 libaries also an open-source Python libary. It is exspecially know for making statistical graphiscs butilt on top of the matplot lib. Seaborn proves high-level interface for plotting informatikve and statitical plots. It is also closely integrates with Pandas data structure what allows easy visualitzion of whole datasets and enables - Plotting of complex relationships - aumtomtac aggregation and statistical estimation (Means, confidence intervals, etc) - Thematic styling and enhanced color palettes. Therefore it was for me a perfect additon to the pandas and Mathplot libaries. [Wt25]

In addition to Matplotlib and Pandas, I also used the open-source Python library **Seaborn**. Seaborn is especially known for creating statistical graphics and is built on top of the Matplotlib library. It provides a high-level interface for drawing informative and attractive statistical plots. Seaborn is closely integrated with Pandas data structures, which makes it easy to visualize entire datasets. This integration enabled me to create complex visualizations with minimal code. Key features of Seaborn include:

- Plotting of complex relationships between multiple variables
- Automatic aggregation and statistical estimation (e.g., means, confidence intervals)
- Thematic styling and enhanced color palettes for better aesthetics

For these reasons, Seaborn was a perfect addition to the Pandas and Matplotlib libraries in my data analysis workflow. [Wt25]

Before I startet with my own Feature selection I took a closer look at the selected HPC and Kernel features from the paper: Enhancing EV Charging Station Security Using A Multi-dimensional Dataset : CICEVSE2024 [EDBF24].

| Event | Description | Backdoor | Crypto | Dos | Recon |
|---|---|---|---|---|---|
| instructions | Number of instructions executed | 214.19 | 943.54 | 316.65 | 130.69 |
| cache-misses | Number of cache misses | 39.92 | 397.06 | 112.83 | 64.70 |
| exc_taken | Exception taken | 86.82 | 61.42 | 169.62 | 44.55 |
| cpu-migrations | CPU Migrations | 842.43 | -100.00 | 1425.70 | 110.05 |
| dTLB-store-misses | Data TLB - Write Misses | 39.56 | 1068.57 | 101.48 | 72.97 |
| l1d_cache_wr | Level 1 data cache access - Write | 143.87 | 121.56 | 321.24 | 113.14 |
| L1-icache-loads | Level 1 instruction cache access - Read | 163.10 | 538.63 | 283.92 | 107.18 |
| l2d_cache_rd | Level 2 data cache - Read | 51.47 | 422.95 | 151.50 | 70.75 |
| mem_access_rd | Data memory access - read | 118.19 | 387.56 | 253.17 | 91.42 |
| mem_access_wr | Data memory access - write | 142.94 | 119.96 | 322.17 | 114.86 |
| kmem_kfree | Kernel memory freeing event | 149.06 | 67.66 | 2087.07 | 187.25 |
| net_dev_xmit | Network device transmission event | 27.42 | 33.79 | 172.88 | 59.64 |
| qdisc_dequeue | Dequeue event | 31.47 | 42.66 | 261.16 | 88.42 |
| raw_syscalls_sys_enter | System call entry (raw) event | 107.66 | 79.70 | 2.35 | 44.13 |
| irq_softirq_raise | Software interrupt - Raised | 90.29 | 65.72 | 1573.65 | 109.45 |
| sched_migrate_task | Task migration event in the scheduler | 670.49 | -100.00 | 1091.74 | 78.18 |
| sched_switch | Task switch event in the scheduler | 42.10 | 39.22 | 332.82 | 24.59 |
| syscalls_sys_enter_close | System call entry for close syscall | 421.15 | 273.72 | 15.58 | 231.63 |
| syscalls_sys_enter_read | System call entry for read syscall | 44.32 | 30.19 | 37.09 | 56.21 |
| syscalls_sys_enter_write | System call entry for write syscall | 119.30 | 35.73 | 57.34 | 76.92 |

Figure 8: Selected HPC and Kernel Features from [EDBF24]

If we took at the timeline of the selected features we can already see some interesting patterns and trends especially for the (icmp-, tcp-, synonymous-) flooding attacks.

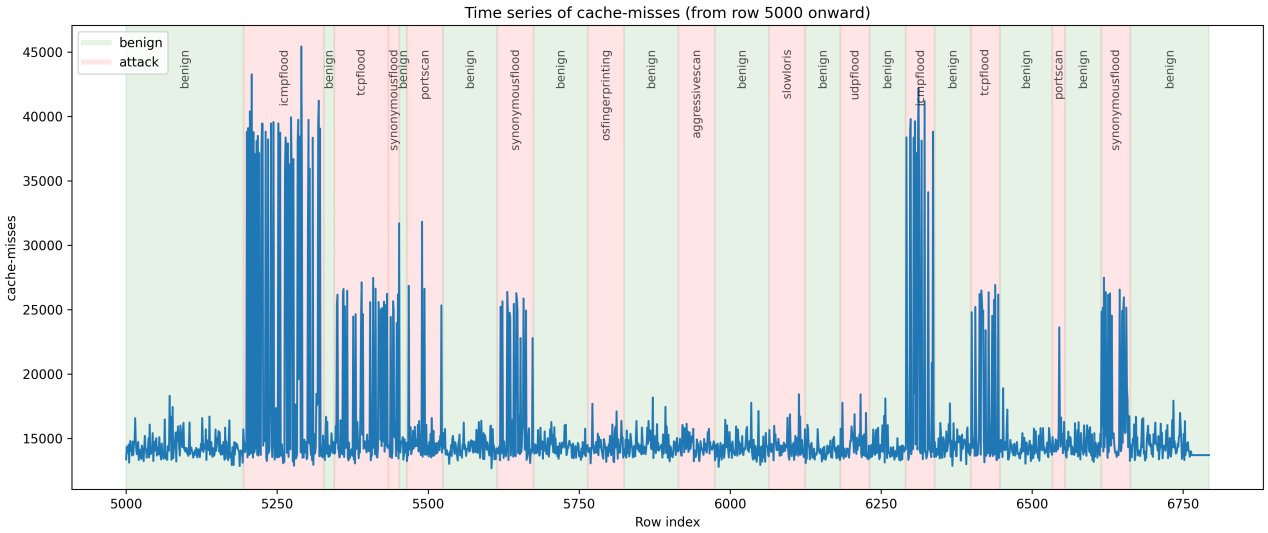


Figure 9: cache-misses timeline

*Cache-misses* Monitors the number of times a data request is not found in the last level cache (LLC)[Per].

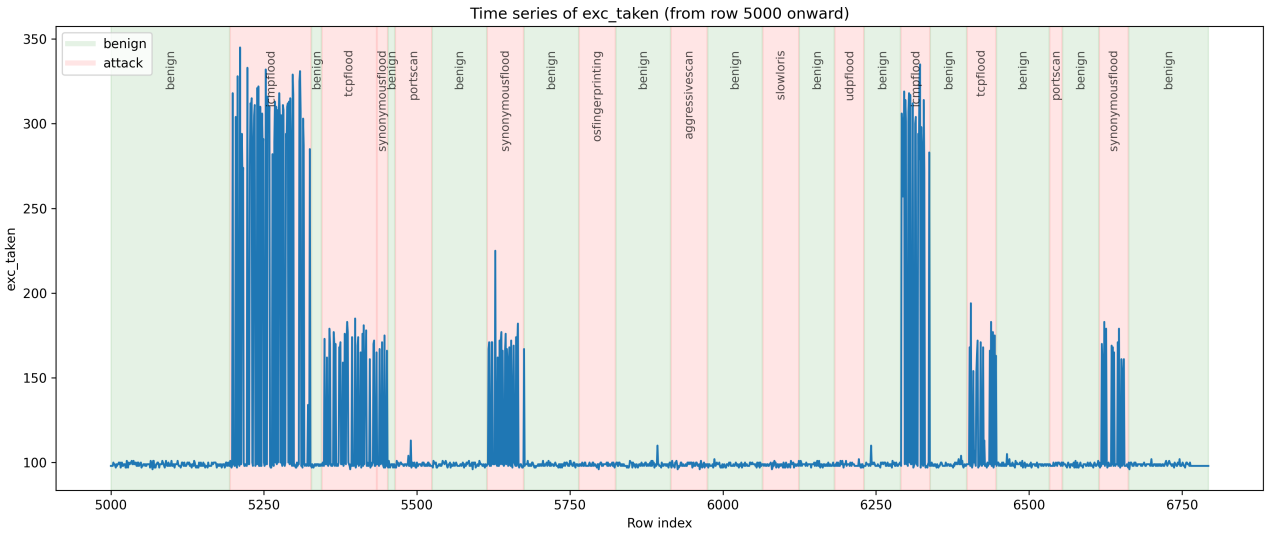


Figure 10: exc-taken timeline

*exc-taken* Tracks the number of exceptions taken by the processor, like interrrupts or faults [Arma].

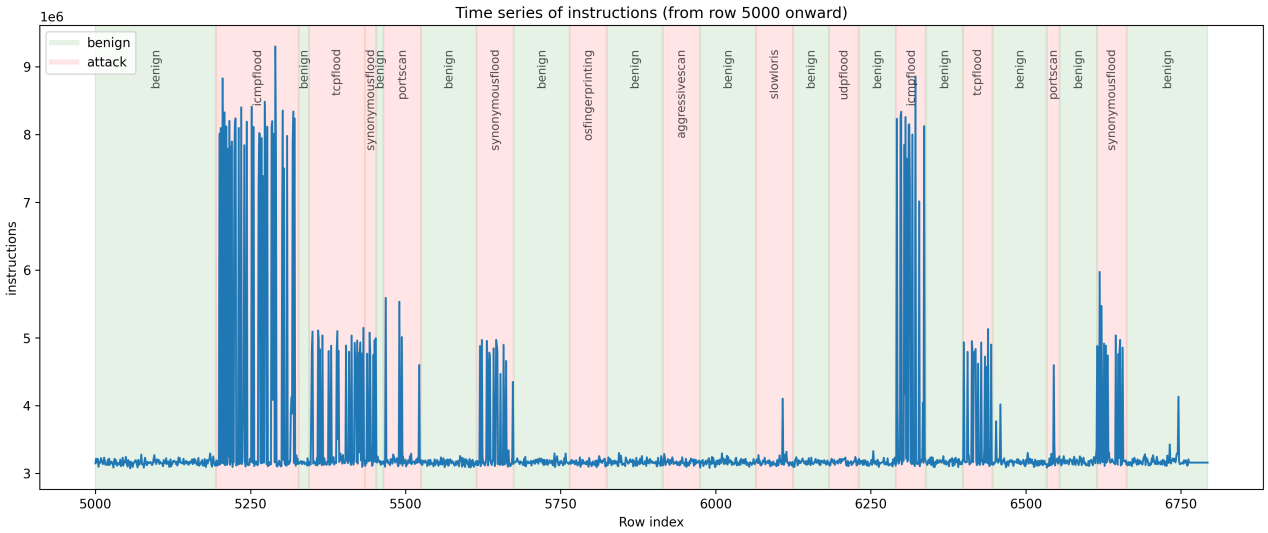


Figure 11: instructions timeline

*instructions* Counts the total number of instructions that have been completely executed by the processor [per].

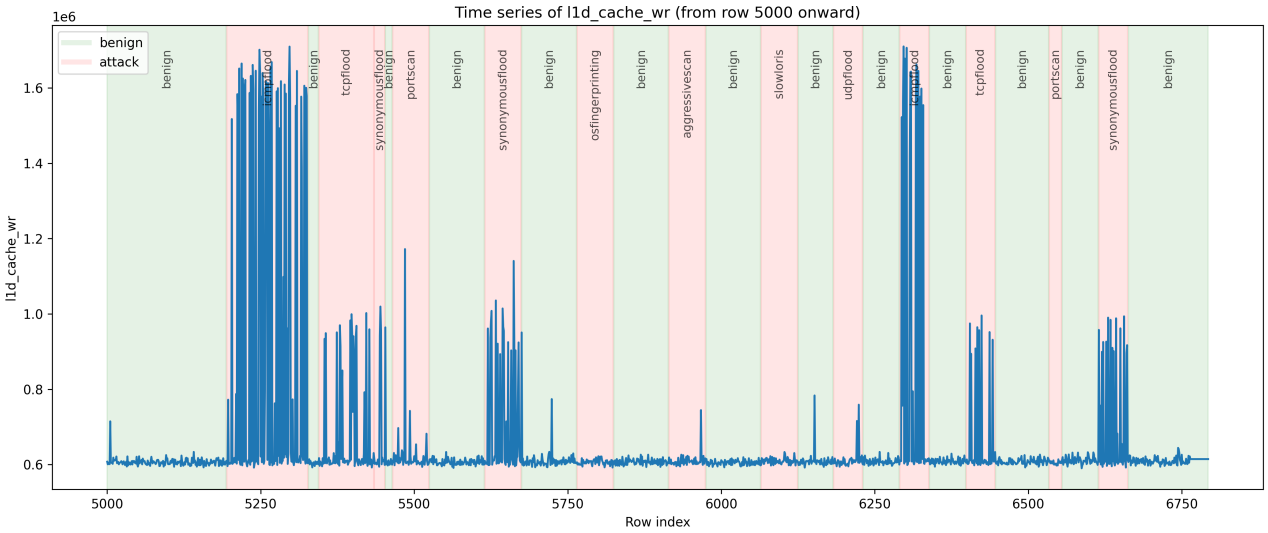


Figure 12: l1d cache wr timeline

*L1d cache wr* Counts every data load, store, or pagewalk that fetches data from ouside the L1 data cache [Armb].

Interestingly, the ICMP flood generates significantly more traffic across almost all measured features, such as *cache misses*, *exceptions taken*, *instructions executed*, and *L1-icache-loads*. The ICMP flood attack is a type of Denial-of-Service (DoS) attack that overwhelms a target system with ICMP Echo Request packets, causing it to become unresponsive.

When compared to other attacks such as the TCP flood or the SYN flood, the ICMP flood

almost doubles the observed values in several performance metrics. This can be explained by the fact that every single ICMP Echo Request must be fully processed and answered by the target system. As a result, this leads to increased memory accesses outside of the CPU caches, as well as more frequent interrupts within the kernel.
The paper *"An Experimental Detection of Distributed Denial of Service Attack in CDX 3 Platform Based on Snort"*[Che+23] also explains that the ICMP flood sends a significantly higher number of Echo Request packets, each of which requires an immediate response from the target system—unlike the TCP flood or SYN flood, which primarily rely on connection queues and do not necessarily require an immediate, full response.

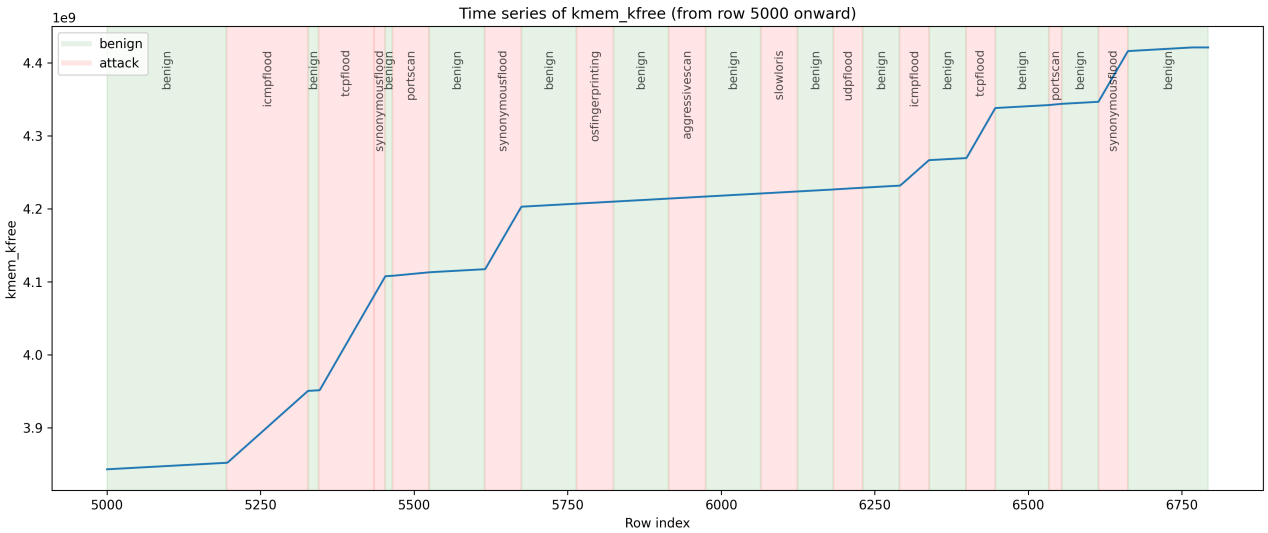


Figure 13: kmem kfree timeline

*kmem kfree* monitors memory free events, specifically tracking when kernel memory objects are released or freed [The].

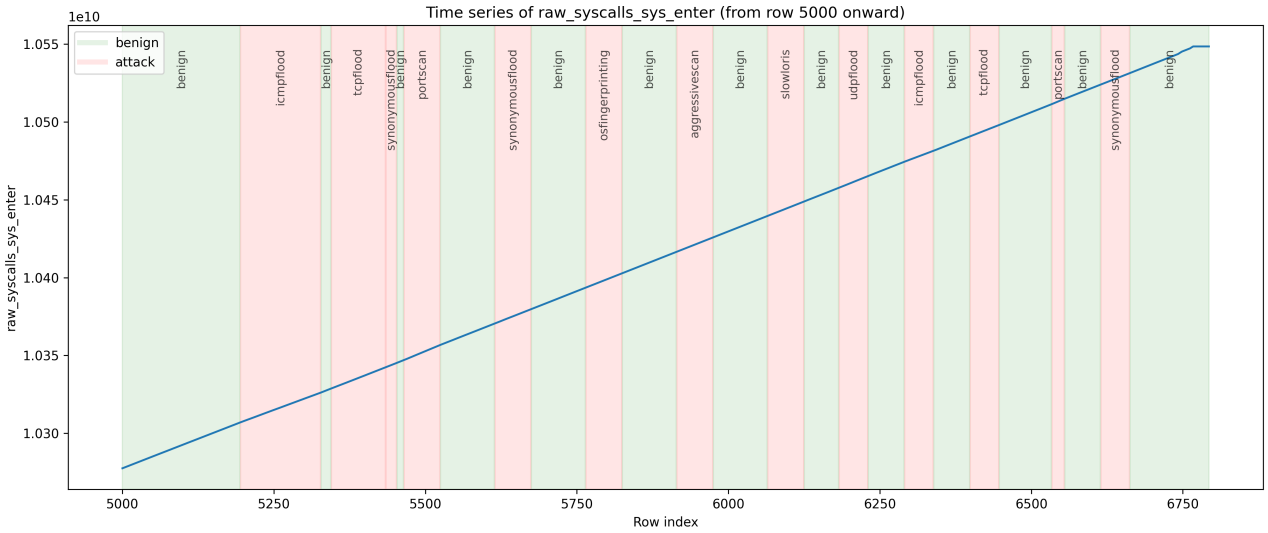


Figure 14: raw syscalls sys enter timeline

*raw syscalls sys enter* captures system call entry (raw) events [EDBF24].

In these graphics, we can observe that *kmem kfree* shows a clearly recognizable increase during ICMP, SYN, and TCP flooding attacks. This makes it a useful and distinguishable feature for detecting such attacks in the later algorithm.
In contrast, the *raw_syscalls_sys_enter* feature exhibits a continuous increase regardless of whether the charging station is under attack or not. This behavior renders it unpredictable in the context of my data and, therefore, not a suitable candidate for the detection algorithm later.

This are just few features of more than 900 features that the paper *Enhancing EV Charging Station Security Using A Multi-dimensional Dataset* [EDBF24] and my own analysis monitored. Later we take a closer look how I selected the most important features for my anomaly detection algorithm.

## 5.2 Comparison of Public and Self-Generated Datasets

The paper *Enhancing EV Charging Station Security Using a Multi-dimensional Dataset* [EDBF24] and my work pursue the same goal: detecting anomalies through the analysis of HCP and kernel events. Therefore, it is essential to compare the underlying datasets on which the machine learning algorithms are trained. Such a comparison enables validation of the respective approaches and helps determine whether the results can be generalized.

We tried to get as close as possible to the environment of the paper *Enhancing EV Charging Station Security Using a Multi-dimensional Dataset*

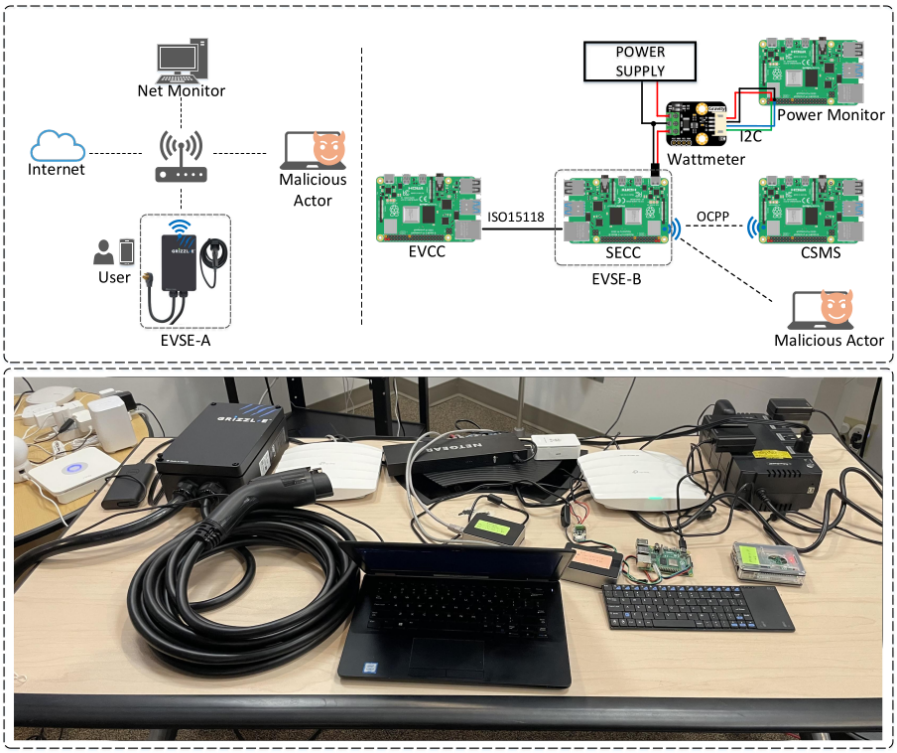


Figure 15: setup EV Charing Station from [EDBF24]

We also used a Raspberry Pi to represent the EVSE-B and employed a wattmeter to monitor power consumption. The combination of HCP events, kernel events, and power consumption measurements will be discussed in future work. As mentioned in Chapter `Data Sources for EVSE Anomaly Detection`, we also used the `perf` tool to monitor the various events.

Therefore, we are able to observe approximately 900 events—similar to the comparison dataset. The complete observation period spans from `2025-07-22T18:00:00` to `2025-07-23T12:52:10`, with a sampling interval of 10 seconds, resulting in approximately 8,000 data rows with 905 columns. There are no missing (NaN) values in the dataset, and all measurements were collected using the `perf` tool, producing logically consistent outputs, as described in the chapter *Data Sources for EVSE Anomaly Detection*.
Features that are distributed across multiple CPU cores (e.g., `cpu=0,1,2,3`) are aggregated using appropriate `perf` flags. This prevents data loss and reduces the number of distinct features, aligning the dataset structure more closely with that of the comparison `CICEVSE2024` dataset. The `CICEVSE2024` dataset consists of the same 905 event types and comprises approximately 8,500 rows. Timestamps are recorded as floating-point numbers representing the time of each measurement. The data was collected using the `perf` tool, ensuring high precision and consistency across all recorded events.
Out of the entire dataset, only four rows contain `NaN` values across all features. These rows can be considered outliers or errors and can easily be excluded during preprocessing. To ensure

comparability with the reference dataset, features distributed across multiple CPU cores (e.g., `cpu=0,1,2,3`) were aggregated using appropriate `perf` flags. While it is not explicitly stated in the documentation of the `CICEVSE2024` dataset whether a similar aggregation was performed, the reduced feature dimensionality suggests that such a step may have been taken. Therefore, this approach was adopted in our dataset to align the structure and avoid data duplication.

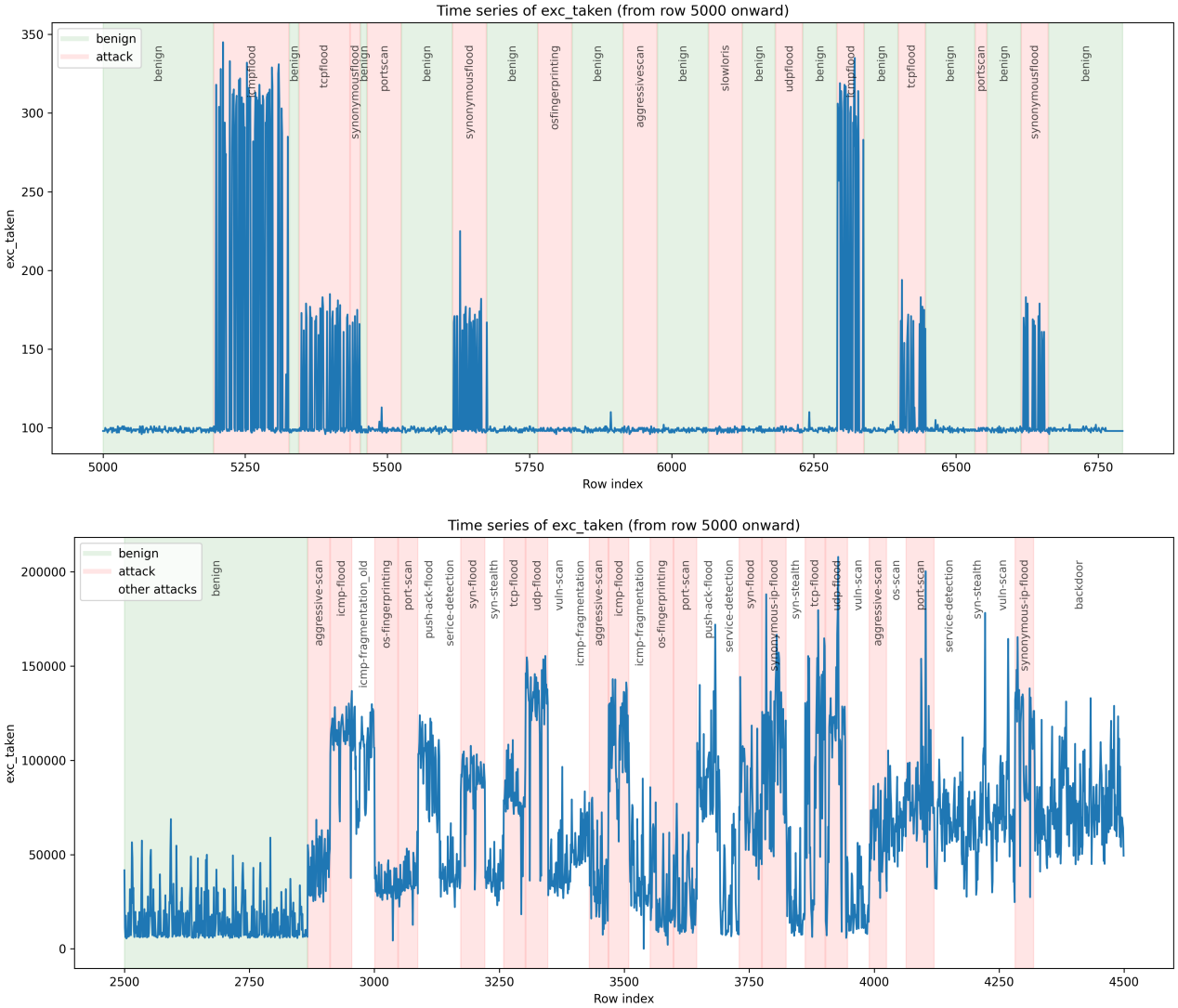


Figure 16: Comparison of `exc-taken` timelines: own dataset (top) and CICEVSE2024 dataset (bottom)

Despite notable differences in the absolute value ranges—likely due to architectural differences, sampling intervals, and system load—the time series of `exc_taken` in both datasets exhibit structurally similar patterns. In both cases, this feature reacts to specific attacks with abrupt value changes especially by the flooding attacks are distinctive spikes, while maintaining stable behavior during benign phases. This confirms `exc_taken` as a sensitive and transferable indicator of anomalous activity in kernel-space, suitable for machine learning-based intrusion

detection approaches.

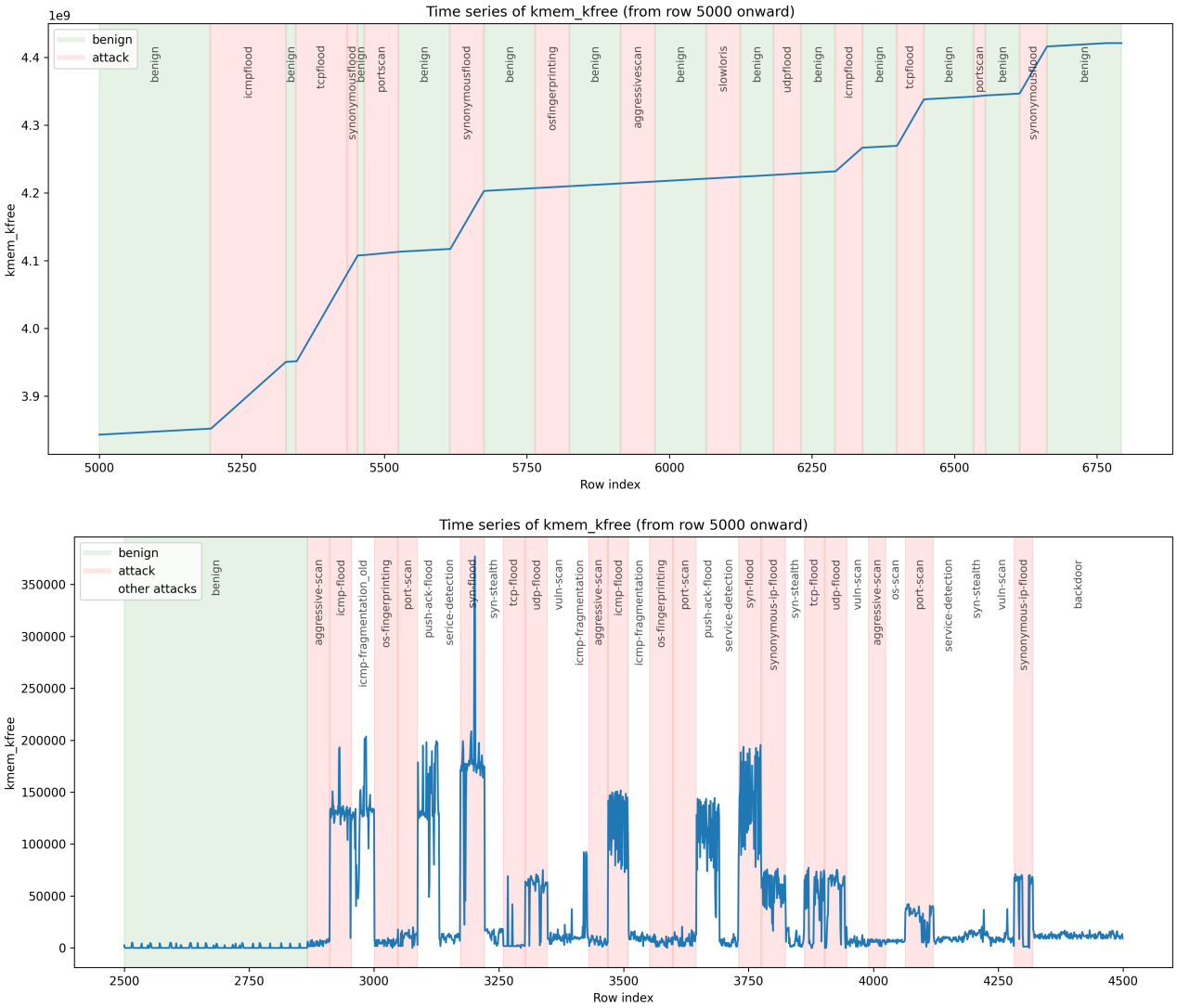


Figure 17: Comparison of `kmem-kfree` timelines: own dataset (top) and CICEVSE2024 dataset (bottom)

We observe similar peaks in both datasets, particularly during the flooding attacks, where the counter for `kmem_kfree` increases noticeably. In our dataset, the resulting curve is cumulative, showing pronounced peaks corresponding to the attack phases. In contrast, the CICEVSE2024 dataset exhibits a more oscillating pattern, which is likely a result of different `perf` measurement flags. Specifically, their approach appears to monitor within a fixed time window, resetting the counter after each interval. This methodological difference leads to the observed oscillatory course in the public dataset, while the cumulative approach in our dataset amplifies peaks and produces a monotonically increasing curve.

Therefore, we have sufficient data to train our models with minimal error values. Moreover, for

the selected features from the paper *Enhancing EV Charging Station Security Using a Multi-dimensional Dataset* [EDBF24], we observe similar peaks and behavioral patterns in both attack and benign scenarios. This indicates a high degree of similarity between the datasets, further supporting the generalizability and validity of the selected features for anomaly detection.

# 6 Feature Selection

## 6.1 Data Preprocessing Steps

As already mentioned, for the `CICEVSE2024` dataset, it was only necessary to remove four rows containing `NaN` values to obtain clean and usable data for the initial processing step.
For the generated dataset from our environment, I first aimed to include as much benign-status data as possible. Therefore, I made a Prometheus PromQL request to retrieve monitoring data from a night when the environment was not used for other tests. The complete script for this query can be found in Listing **??**.

```
1
2       PROMETHEUS_URL = "http://192.168.1.25:9090"
3       STEP = "10"  # seconds
4       CHUNK_HOURS = 2
5
6       start_time = datetime.fromisoformat("2025-07-22T18:00:00.000000")
7       end_time = datetime.fromisoformat("2025-07-23T14:40:00.000000")
8
9       def main():
10      chunks = build_chunks(start_time, end_time, CHUNK_HOURS)
11      data = defaultdict(dict)
12      timestamps = set()
13      values_dict = {}
14      metric_query_map = {}
15
16      for feat in FEATURES:
17          prom_name = feat.replace("-", "_")
18          url = f"{PROMETHEUS_URL}/api/v1/query"
19          hit = False
20
21          # 1. sum(prom_name) abfragen
22          query = f"sum({prom_name})"
23          params = {'query': query}
24          try:
25              resp = requests.get(url, params=params)
26              resp.raise_for_status()
27              results = resp.json()['data']['result']
28              if results:
29                  value = float(results[0]['value'][1])
30                  values_dict[feat] = value
31                  metric_query_map[feat] = query
32                  hit = True
33          except Exception:
34              pass
```

Listing 3: Python script that defines the timesteps and time range for data scraping and performs the request from the `prometheus_URL`

Then, I initiated the attacks to monitor the behavior of the HCP and kernel events in our environment under attack conditions. The complete script can be found in Listing **??**.

```
commands = [
("Slowloris", ["python3", "slowloris.py", "192.168.1.25", "-s", "500"]),
("UDP Flood", ["sudo", "hping3", "--flood", "--udp", "-p", "80", "192.168.1.25"]),
("ICMP Flood", ["sudo", "hping3", "--flood", "-1", "192.168.1.25"]),
("TCP Flood", ["sudo", "hping3", "--flood", "-p", "80", "192.168.1.25"]),
("Synonymous Flood", ["sudo", "hping3", "--flood", "--rand-source", "-p", "80", "192.168.1.25"]),
("Portscan", ["bash", "-c", "for i in {1..1000}; do sudo nmap -sT 192.168.1.25; sleep 1; done"]),
("OS Fingerprinting", ["bash", "-c", "for i in {1..1000}; do sudo nmap -O 192.168.1.25; sleep 1; done"]),
("Aggressive Scan", ["bash", "-c", "for i in {1..1000}; do sudo nmap -A 192.168.1.25; sleep 1; done"]),
]

def log_msg(msg):
with open(LOG_FILE, "a") as f:
    f.write(msg + "\n")

while True:
    for name, cmd in commands:
        start_time = datetime.now()
        print(f"Starte {name}: {' '.join(cmd)}")
        log_msg(f"Start: {start_time.strftime('%Y-%m-%d %H:%M:%S')} - Angriff: {name} - Befehl: {' '.join(cmd)}")
        try:
            proc = subprocess.Popen(cmd)
            proc.wait(timeout=RUN_TIME)
            note = "Erfolgreich beendet"
        except subprocess.TimeoutExpired:
            proc.terminate()
```

Listing 4: Python script that executes the commands during the defined period and logs the timestamps of the different attacks

With the generated log file, it was possible to insert the different attacks with the correct time periods into the `dataset.csv` file for use in training the machine learning models and do a correct data analysis.

```
# alle angaben sind schon auf kleinschreibung und das richtige format angepasst
intervals = [
```

```
    ("2025-07-23T06:52:35.000", "2025-07-23T07:07:35.000", "slowloris"),
    ("2025-07-23T07:27:35.000", "2025-07-23T07:42:35.000", "udpflood"),
    ("2025-07-23T08:25:45.000", "2025-07-23T08:47:50.000", "icmpflood"),
    ("2025-07-23T08:50:49.000", "2025-07-23T09:05:49.000", "tcpflood"),
    ("2025-07-23T09:05:49.000", "2025-07-23T09:08:49.000", "
       synonymousflood"),
    ("2025-07-23T09:10:44.000", "2025-07-23T09:20:44.000", "portscan"),
    ("2025-07-23T09:35:44.000", "2025-07-23T09:45:44.000", "
       synonymousflood"),
    ("2025-07-23T10:00:44.000", "2025-07-23T10:10:44.000", "
       osfingerprinting"),
    ("2025-07-23T10:25:44.000", "2025-07-23T10:35:44.000", "
       aggressivescan"),
    ("2025-07-23T10:50:44.000", "2025-07-23T11:00:44.000", "slowloris"),
    ("2025-07-23T11:10:21.000", "2025-07-23T11:18:21.000", "udpflood"),
    ("2025-07-23T11:28:21.000", "2025-07-23T11:36:21.000", "icmpflood"),
    ("2025-07-23T11:46:21.000", "2025-07-23T11:54:21.000", "tcpflood"),
    ("2025-07-23T12:08:55.000", "2025-07-23T12:12:21.000", "portscan"),
    ("2025-07-23T12:22:21.000", "2025-07-23T12:30:21.000", "
       synonymousflood"),
]

for start, end, value in intervals:
    maske = (df["timestamp"] >= start) & (df["timestamp"] <= end)
    df.loc[maske, "attack"] = value
    df.loc[maske, "Label"] = "attack"
```

Listing 5: Command to insert the attack labels into the dataset using the Python library `pandas`

With these preprocessing steps, a meaningful exploratory data analysis was possible, allowing us to proceed with key feature selection to train the machine learning models for optimal results. This step is essential, as most models do not perform well with 900 or more features. This is also demonstrated in [EDBF24], where the best results were achieved using only 11 to 20 key features.
Furthermore, using too many features would lead to performance issues on the Raspberry Pi in the live system, as it would need to query and process values for up to 900 features every 5 to 60 seconds.

## 6.2 Feature Selection Using Decision Trees

Feature selection is a crucial aspect in the preparation of modern deep learning algorithms. However, identifying the most effective method that also performs well in a live environment—such as an EVSE system with limited hardware resources—remains a challenge.
The paper *"Enhancing EV Charging Station Security Using a Multi-dimensional Dataset" [EDBF24]* employed Principal Component Analysis (PCA) to reduce 900 input features for use in an LSTM

model. We also experimented with PCA, but due to the nature of this method, all 900 original features must still be requested in the live environment in order to compute the transformed principal components. Executing the required PromQL queries for all 900 features caused the CPU usage on our Raspberry Pi setup to average nearly 50 %, which makes this approach unsuitable for real-time deployment.
Following this limitation, we were compelled to explore alternative feature selection techniques that could offer high performance without requiring access to the full feature set. The paper *"Comparative Analysis of Feature Selection Techniques for LSTM-Based Network Intrusion Detection Models" [Kot24]* evaluated multiple feature selection strategies to determine the most effective combinations for LSTM-based algorithms. According to their findings, the

| Model Configuration | Accuracy | F1 Score |
|---|---|---|
| LSTM + Without Feature selection | 85% | 0.872 |
| LSTM + PCA | 76% | 0.821 |
| LSTM + Univariate Feature Selection | 78% | 0.836 |
| LSTM + Recursive Feature Elimination | 78% | 0.837 |
| LSTM + L1- Based Feature Selection | 77% | 0.829 |
| LSTM + Select Best Feature Selection | 83% | 0.871 |
| LSTM + Tree Based Feature Selection | 82% | 0.857 |
| LSTM + Sequential Feature Selection | 69% | 0.736 |

Figure 18: Comparison of feature selection methods for LSTM accuracy [Kot24].

LSTM model without any feature selection achieved the highest accuracy at 85 %. However, as previously mentioned, using all 900 features is not feasible in our hardware-constrained environment. The next best accuracies were achieved using LSTM combined with either the Select Best Feature Seleciton method or Tree-Based feature selection, with only a 1 % accuracy drop and an F1-score difference of just 0.014.
Given this background, we decided to adopt the Decision Tree algorithm for feature selection. This allows us to identify the most relevant features while maintaining a high level of accuracy, and keeping resource usage within our system s constraints.

With this information, I created a Decision Tree to select the most relevant features for real-time detection in our EVSE system. I used the `DecisionTreeClassifier` from the `sklearn.tree` module, which is easy to use and integrates well with common data science workflows.
(muss ich das begürnden?)

In the following script, you can see how I trained our preprocessed dataset using binary labels (benign = 0, attack = 1). To prevent data leakage, I dropped the columns `'Label'`, `'timestamp'`, and `'attack'` before training.

```
1 import pandas as pd
```

```
from sklearn.tree import DecisionTreeClassifier

# Load preprocessed custom dataset
df = pd.read_parquet('preprocessed_custom_data.parquet')

# Convert categorical labels to binary format: benign = 0, attack = 1
df['Label'] = df['Label'].replace({'benign': 0, 'attack': 1})

# Define features (X) and target variable (y)
# Drop metadata and redundant columns
X = df.drop(columns=['Label', 'timestamp', 'attack'])
y = df['Label']

# Initialize and train a Decision Tree classifier for feature selection
clf = DecisionTreeClassifier(random_state=42)
clf.fit(X, y)
```

Listing 6: Training a Decision Tree Classifier for feature selection using a binary-labeled dataset

This approach allowed us to significantly reduce the number of input features while preserving high detection accuracy and runtime performance.

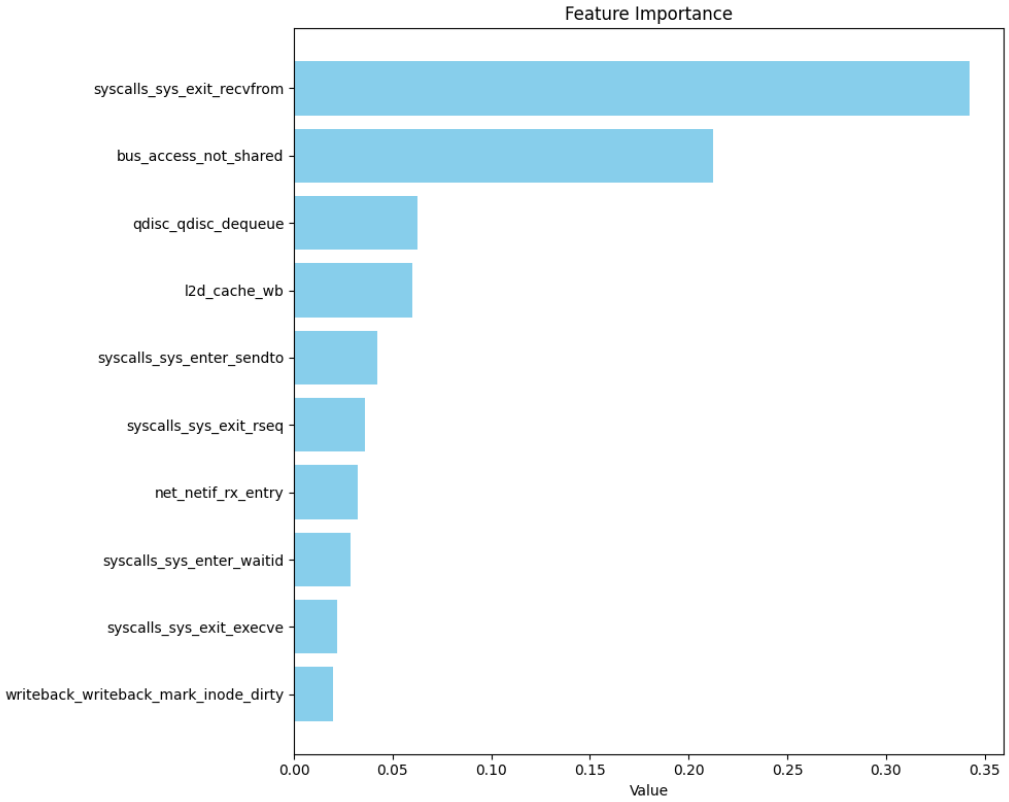


Figure 19: Top 10 most important features selected by the Decision Tree.

TODO explain here the 3 most important features what the are..

Using the 20 most important features identified by the Decision Tree, I created a correlation matrix to examine feature redundancy.

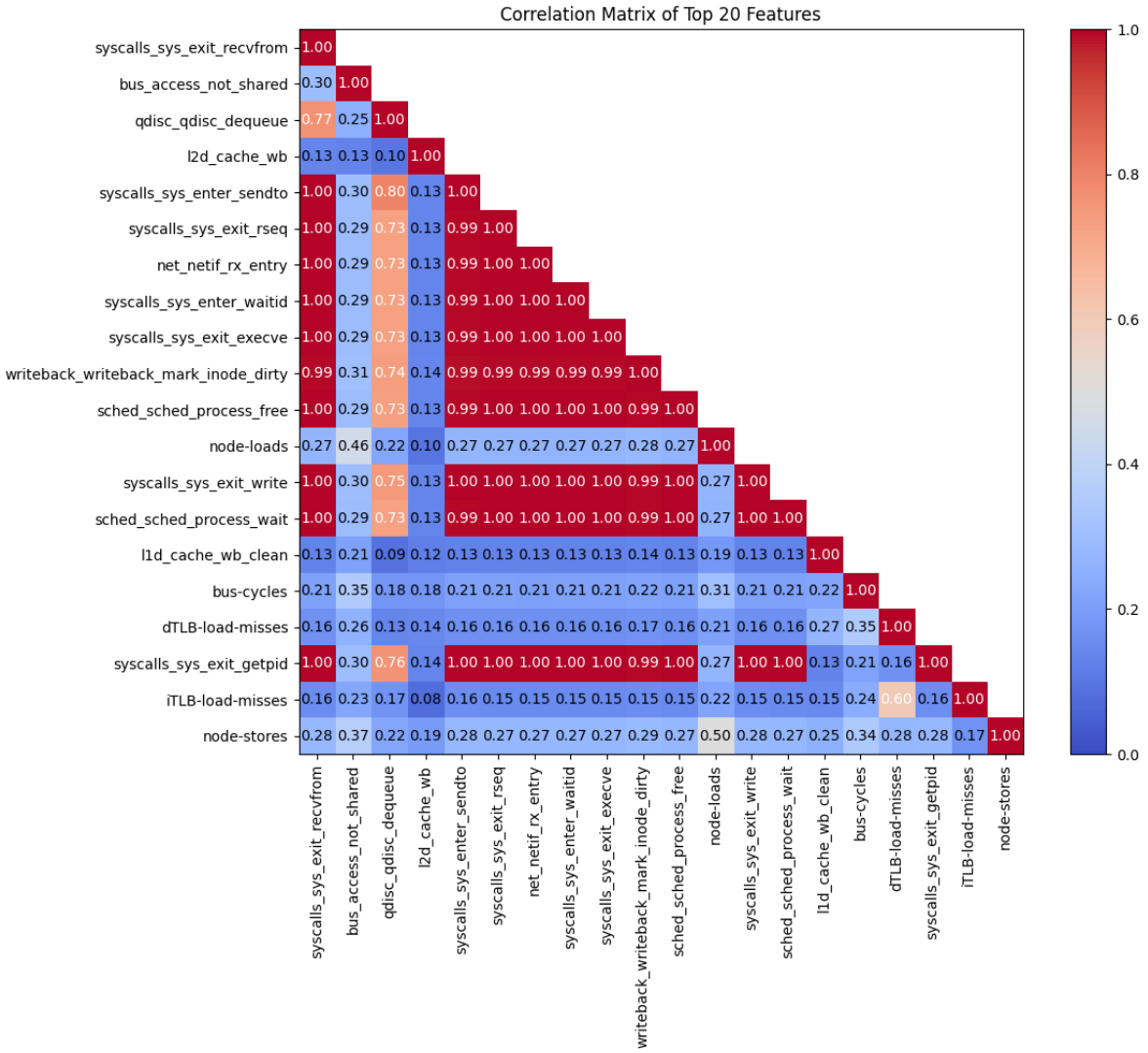


Figure 20: Correlation matrix after the first Decision Tree run.

However, a closer look at the correlation matrix reveals that many of these features are highly correlated with one another, reducing their individual significance in model training.
To increase the diversity of the selected features and avoid redundancy, I removed highly correlated features (correlation coefficient $> 0.99$). When two features were strongly correlated, I retained only the one with the higher importance score.

```
# Remove strongly correlated features (>0.99),
# keeping the one with higher importance

```

```
# X: DataFrame containing feature columns
# importances: Pandas Series with feature names as index

# Compute absolute correlation matrix
corr_matrix = X.corr().abs()
# Extract upper triangle (excluding diagonal)
upper_tri = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).
    astype(bool))

to_drop = set()

# Iterate over features
for col in upper_tri.columns:
    high_corr_feats = upper_tri.index[upper_tri[col] > 0.99].tolist()
    for feat in high_corr_feats:
        # Drop the feature with lower importance
        if importances[feat] < importances[col]:
            to_drop.add(feat)
        else:
            to_drop.add(col)

# Drop selected features
X_reduced = X.drop(columns=to_drop)
```

Listing 7: Dropping highly correlated features based on importance scores

After this filtering step, I retrained the Decision Tree using the reduced feature set. The resulting top features were more diverse and better distributed, providing a more robust foundation for training the LSTM model in the next stage.

If we take a look at the timeline and changes from the new selected features we can observe that not just flooding attacks should be detect sondern auch port scans what gaves us the possbility das wir breiter aufgestellt sind.

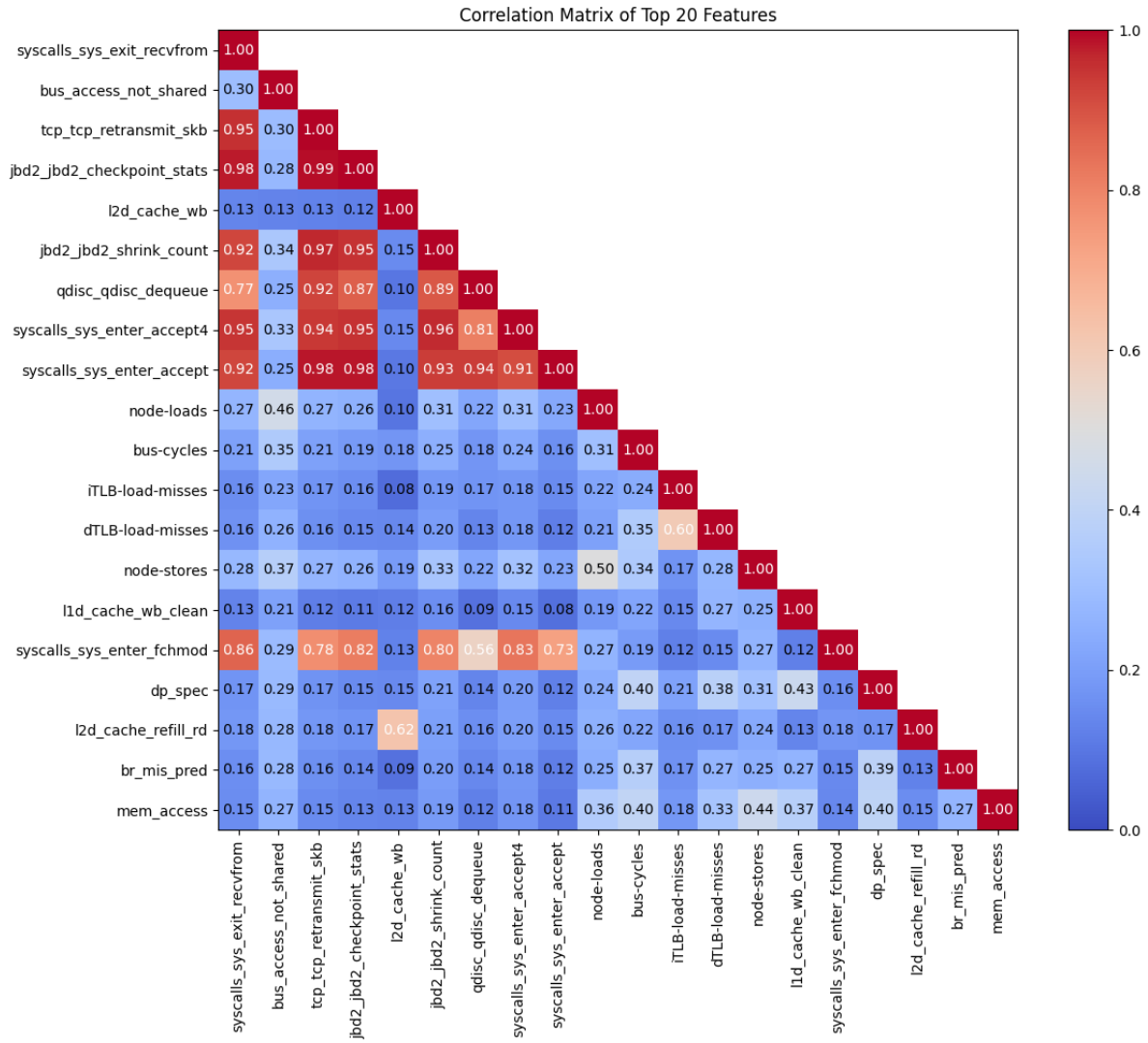


Figure 21: Top 20 most important features selected by the Decision Tree after dropping high correalted features.

# 7 LSTM Architecture: Structure, Training, and Optimization

## 7.1 LSTM Architecture and Preprocessing

A foundational understanding of the structure and principles of Long Short-Term Memory (LSTM) networks has already been provided in Chapter 2_Fundamentals. In this section, I will delve deeper into the specific architecture of the LSTM used in this work and highlight the importance of time-step-based data analysis in this context.

Time-step analysis offers a major advantage of LSTMs over other deep neural network (DNN) models, particularly in the detection of cyber attacks or anomalies. Most such events do not occur abruptly. Instead, they often emerge gradually over time [Rah+25]. LSTM networks are

specifically designed to recognize temporal patterns and sequential dependencies, making them especially well-suited for identifying these slow-developing behaviors.

In order to detect time-dependent attacks effectively, the training data must be well-organized so that the models can learn time-relevant patterns—both during training and later in the live system. Therefore, the preprocessing steps used to split the data into training and test sets must be carefully structured and should not be randomized.
At this stage, the data has generally already been cleaned, as described in Chapter 6_Feature Selection.
In my case, I split the dataset into 10 chronological blocks. Within each block, the data is sorted by time steps. Then I combined 7 of these blocks to the training dataset and used the remaining 3 for the test dataset. I ensured that both sets had approximately the same distribution of the labels `benign` and `attack`, to prevent class imbalance.
The following listing shows the exact implementation of the data-splitting and label distribution verification process:

```
1
2  import pandas as pd
3
4  # Load the preprocessed dataset
5  df = pd.read_parquet('preprocessed_custom_data.parquet')
6
7  # Split the dataset into 10 time-ordered blocks
8  block_size = len(df) // 10
9  blocks = [df.iloc[i * block_size:(i + 1) * block_size] for i in range(9)]
10 blocks.append(df.iloc[9 * block_size:])  # Last block takes the remaining
       data
11
12 # Print class distribution in each block
13 label_counts = [block['Label'].value_counts() for block in blocks]
14 for i, counts in enumerate(label_counts):
15     print(f'Block {i + 1}:', dict(counts))
16
17 # Use blocks 2, 5, and 8 as test data, the rest as training data
18 test_blocks = [blocks[1], blocks[4], blocks[8]]
19 train_blocks = [block for i, block in enumerate(blocks) if i not in [1,
      4, 8]]
20
21 # Concatenate the blocks to form final datasets
22 test_df = pd.concat(test_blocks).reset_index(drop=True)
23 train_df = pd.concat(train_blocks).reset_index(drop=True)
24
25 # Save the resulting datasets as Parquet files
26 test_df.to_parquet('test_block.parquet')
27 train_df.to_parquet('train_block.parquet')
28
```

```
# Reload saved Parquet files (for verification or further use)
train_df = pd.read_parquet('train_block.parquet')
test_df = pd.read_parquet('test_block.parquet')

# Function to compute label distribution percentages
def label_percentages(df):
    return (df['Label'].value_counts(normalize=True) * 100).round(2)

# Display label distribution
print("Train Set:")
print(label_percentages(train_df))

print("\nTest Set:")
print(label_percentages(test_df))
```

Listing 8: Block-wise time-ordered split of the dataset for training and testing, including class distribution balancing

```
# Function to compute label distribution percentages
def label_percentages(df):
    return (df['Label'].value_counts(normalize=True) * 100).round(2)

# Display label distribution
print("Train Set:")
print(label_percentages(train_df))

print("\nTest Set:")
print(label_percentages(test_df))
```

```
Train Set:
Label
benign    86.82
attack    13.18
Name: proportion, dtype: float64

Test Set:
Label
benign    84.93
attack    15.07
Name: proportion, dtype: float64
```

Figure 22: Balanced label distribution between the training and test datasets.

## 7.2 Parameter Configuration and Hyperparameter Optimization

In this Chapter we take a closer look at the choosen structure from the LSTM architecture and explain it.

```
from config_feature_test import FEATURES, LABEL
import pandas as pd
```

```
from sklearn.preprocessing import StandardScaler

# Load training and test datasets
train_df = pd.read_parquet('train_block.parquet')
test_df = pd.read_parquet('test_block.parquet')
```

Listing 9: Loading training and test datasets, and preparing feature and label configuration

In the text code listing we label the data in the dataset and scale it from 0 to 1 this is espacially in the LSTM model important because the unscaled data can not be readed well from the model.

In the code listing 10, we label the data by converting the class labels from strings ("benign", "attack") into binary values (0 and 1), and we scale all feature values using the `StandardScaler`. This scaling step is crucial for training an LSTM model effectively.
Unscaled data can lead to numerical instability and poor convergence in neural networks. LSTMs are particularly sensitive to the scale of input features, because they use sigmoid (explained later in detail) activation functions, which operate best when input values are within a small, centered range—typically between -1 and 1. If the features have very different magnitudes, such as bytes transmitted (e.g. in thousands) versus CPU usage (e.g. in fractions), the model may focus disproportionately on features with larger numeric ranges, leading to biased learning and suboptimal performance [Coo+17].
By standardizing the input features (i.e., transforming them to have zero mean and unit variance), we ensure that the LSTM model can learn temporal dependencies more effectively and converge faster during training.

```
# Convert labels to binary values: 'attack' -> 1, 'benign' -> 0
train_df[LABEL] = train_df[LABEL].map({'attack': 1, 'benign': 0})
test_df[LABEL] = test_df[LABEL].map({'attack': 1, 'benign': 0})

# Separate features and labels
X_train = train_df[FEATURES]  # Select defined features for training
y_train = train_df[LABEL]     # Select the corresponding labels
X_test = test_df[FEATURES]    # Select same features for testing
y_test = test_df[LABEL]       # Select the corresponding labels

# Scale features using StandardScaler (fit on training data only)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)  # Fit and transform on
    training data
X_test_scaled = scaler.transform(X_test)        # Transform test data
    using same scaler
```

Listing 10: Preprocessing: Label encoding, feature selection and scaling for LSTM training

It was also necessary to train the LSTM model with time sequences in order to fully leverage

the advantages of LSTM architectures. Since LSTMs are specifically designed to recognize temporal patterns, providing input in sequential form significantly improves performance. Additionally, it is important that each sequence is labeled consistently as either *benign* or *attack*. This avoids ambiguous target values during training and ensures that the model can learn clear and unambiguous patterns for classification.

```
import numpy as np

def create_sequences(X, y, window_size=10):
    X_seq = []
    y_seq = []
    for i in range(len(X) - window_size + 1):
        label_window = y[i:i+window_size]

        # Only keep sequences where all labels in the window are the same
        if np.all(label_window == label_window[0]):
            X_seq.append(X[i:i+window_size])      # Add sequence window of features
            y_seq.append(label_window[0])         # Add corresponding consistent label
    return np.array(X_seq), np.array(y_seq)
```

Listing 11: Create labeled sequences from time-series data for LSTM training

The following listing is particularly interesting, as it highlights several important configuration choices that warrant further discussion.

| Component | What it is | Why it is used |
|---|---|---|
| **Dropout** | A regularization technique that randomly sets a fraction of input units to zero during training. | Dropout reduces overfitting by randomly disabling parts of the model during training, which leads to a more robust and generalized model. |
| **Sigmoid** | An activation function that outputs values between 0 and 1. | Because it is ideal for binary classification (benign, attack) by producing probabilities for class 0 or 1. |
| **Binary Cross-Entropy** | A loss function for binary classification tasks. | Measures the information loss between predicted probabilities and true binary labels, reflecting both class separation and probability calibration. |
| **Adam** | An optimization algorithm that combines momentum and adaptive learning rates. | Enables fast convergence and performs well with noisy and sparse gradients by adaptively scaling learning rates for each parameter. |

Table 5: Explanation of key components used in the LSTM model configuration.[Aro+21][Sae][Ram+18][KB15]

```python
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout

# Function to create a stacked LSTM model
def create_lstm_model(num_features):
    model = Sequential([
        # First LSTM layer (returns full sequences for stacking)
        LSTM(30, input_shape=(10, num_features), return_sequences=True),
        Dropout(0.2),  # Dropout to prevent overfitting
        # Second LSTM layer (only returns final output)
        LSTM(15),
        Dropout(0.2),  # Additional regularization
        # Output layer for binary classification
        Dense(1, activation='sigmoid')
    ])

    # Compile model with binary crossentropy loss and accuracy metric
    model.compile(
        loss='binary_crossentropy',
        optimizer='adam',
        metrics=['accuracy']
```

```
    )
    return model

# Create model with number of selected input features
model = create_lstm_model(len(FEATURES))

# Print model summary
model.summary()

# Print number of selected features
print(len(FEATURES))
```

Listing 12: LSTM model definition for time-series classification

do I have to explain batch_size. Hat ja nicht wirklich einlfuss auf das trainingsergebnis.
In addition, it should be noted that an early stopping function can be helpful in some cases instead of using a fixed number of 20 epochs. However, for simplicity, I do not discuss or explain the additional code in this work.

```
# Train the LSTM model using the training sequences
history = model.fit(
    X_train_seq,         # Input training sequences
    y_train_seq,         # Corresponding labels
    epochs=20,           # Number of times the model will iterate over the
        entire training data
    batch_size=32,       # Number of samples processed before the model
       updates
    validation_data=(X_test_seq, y_test_seq),  # Data for evaluating the
       model during training
    verbose=1            # Displays progress bar and training metrics
)
```

Listing 13: Training the LSTM model with time-sequenced data

After gaining an understanding of how we designed the LSTM architecture for our use case and the meaning of its various functions, we will review the results in the next chapter.

# 8 Analysis and Evaluation

## 8.1 Experimental Setup and Evaluation Metrics

As mentioned in Chapter 7.1 *LSTM Architecture and Preprocessing*, the data was split into two datasets: one for training and one for testing. In the following figures, we analyze the trained model using the test dataset, which is also included in the appendix.

At first we tested the lstm architecutre in the test dataset where we took befor a closer look, that the distribution of attack an benign data is simular to the train data. the result where ploted with an confusion matrix.
In the following, we tested the LSTM model in the live system too. Using the following scripts:

```
csv_file = 'monitor_lstm_predictions.csv'
header = ['timestamp'] + FEATURES + ['benign_prob', 'attack_prob', '
    prediction']

# Function to query a single metric value from Prometheus
def get_metric_value(prom, feat):
    base_feat = feat.replace('-', '_')

    # Try without "perf_" prefix
    try:
        data = prom.get_current_metric_value(metric_name=f"sum({base_feat
            })")
        df = MetricSnapshotDataFrame(data)
        if not df.empty and 'value' in df.columns:
            return float(df.iloc[0]['value'])
    except Exception:
        pass

    # Try with "perf_" prefix as fallback
    try:
        data = prom.get_current_metric_value(metric_name=f"sum(perf_{
            base_feat})")
        df = MetricSnapshotDataFrame(data)
        if not df.empty and 'value' in df.columns:
            return float(df.iloc[0]['value'])
    except Exception:
        pass

    return np.nan
```

Listing 14: Create CSV file and define the correct PromQL query

```
while True:
    # 1. Query metric values
```

```
    values_array = [get_metric_value(prom, feat) for feat in FEATURES]

    # 2. Log to CSV
    row = [datetime.now().isoformat()] + values_array + [benign_prob,
        attack_prob, prediction]
    with open(csv_file, 'a', newline='') as f:
        csv.writer(f).writerow(row)

    time.sleep(5)
```

Listing 15: Get metric values and log them into the CSV file

At the same time, I started the attack script—just [listinging 17] as used before—to automate the attacks and generate a synchronized timestamp log. This allows later combination into a single CSV file, making the exploratory data analysis more effective. You can finde this dataset also attached under the name: "monitor_lstm_predictions_livedata.parquet".

## 8.2 Results of Anomaly Detection Using the Machine Learning Model

In the chapter *Exploratory Data Analysis*, we also took a closer look at several events that are mentioned in the paper [EDBF24]. These events were well suited for comparison with our dataset. During the analysis, we observed that the highest variance occurred during flooding attacks, while port scans showed almost no changes in the selected events.
Now, we also examine the events identified as key features by the decision tree algorithm to determine whether we can uncover additional findings.

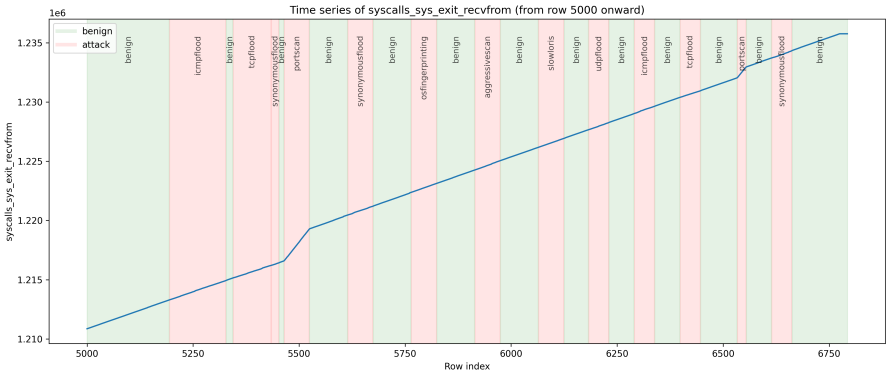


Figure 23: syscalls_sys_exit_recvfrom timeline


Figure 24: bus_access_not_shared timeline

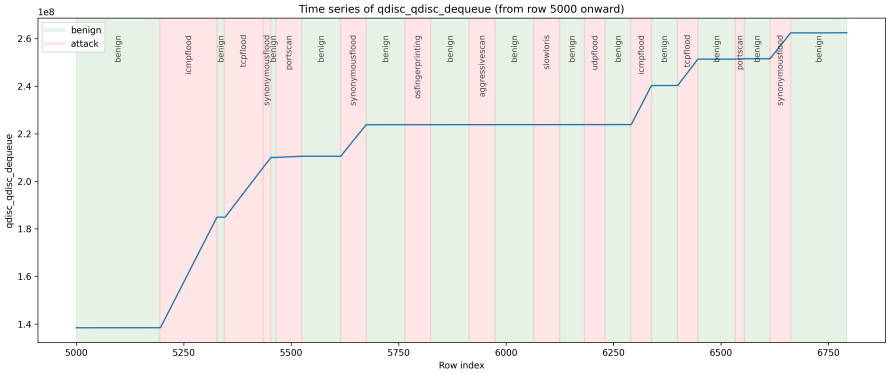


Figure 25: qdisc_qdisc_dequeue timeline

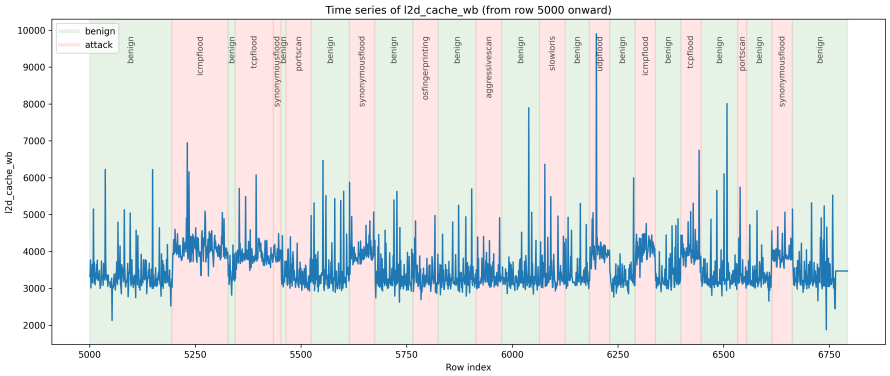


Figure 26: l2d_cache_wb timeline

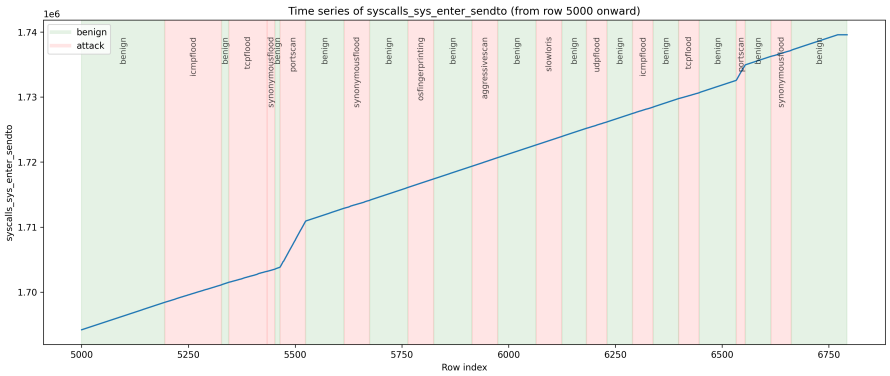


Figure 27: syscalls_sys_enter_sendto timeline

When we examine the key features, we can clearly observe that the highest peaks occur during flooding attacks, as previously identified. Interestingly, two of the key features also show increased activity during port scan attacks. This leads to the insight that these features could potentially be used to detect such attacks as well.
When we now test our LSTM algorithm on the test dataset, we observe exactly this expected result.

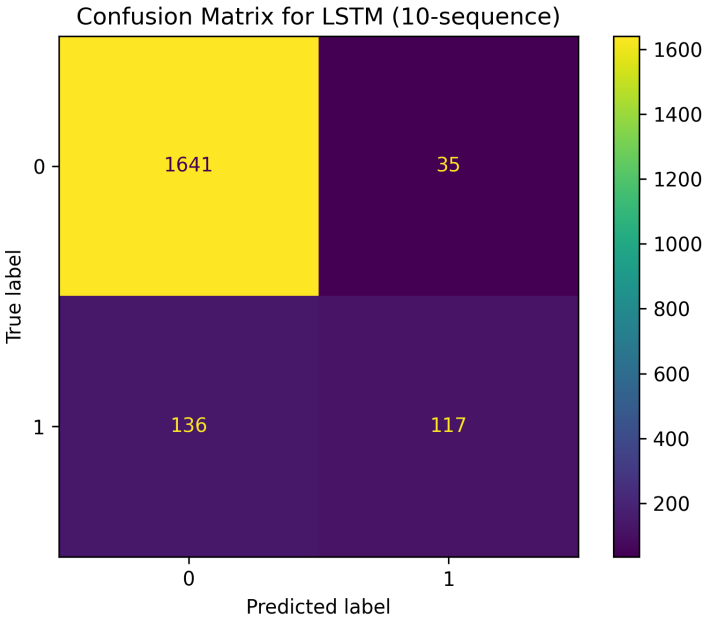


Figure 28: Confusion matrix

| attack | misclassified | total |
|---|---|---|
| osfingerprinting | 46 | 51 |
| aggressivescan | 39 | 51 |
| slowloris | 35 | 37 |
| portscan | 16 | 51 |
| synonymousflood | 0 | 60 |
| tcpflood | 0 | 3 |

Figure 29: Falsely labeled attacks

The algorithm has the most difficulty detecting the OS fingerprinting, aggressive scan, and Slowloris attacks. The port scan attack, however, was detected in most cases—35 out of 51.
In the following we testet the lstm algorithm in the live enviorment, too with simular results.

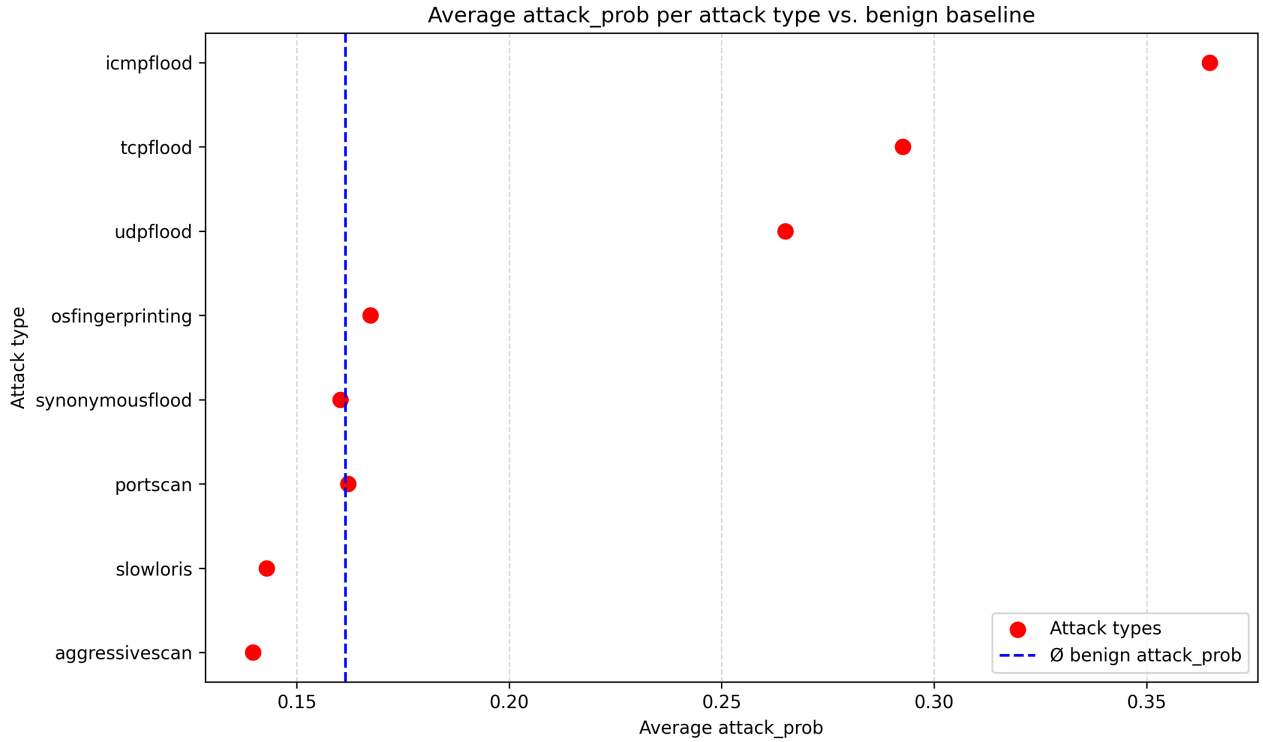


Figure 30: Average `attack_prob` values per attack type compared to the benign baseline. Flooding attacks show significantly higher average probabilities than the benign reference, while slow and scanning attacks remain close to or even below it.

**Attack detection confidence analysis.** Figure 30 shows the average `attack_prob` values for different attack types, as estimated by the classifier, in comparison to the average value observed during benign traffic. The vertical dashed line represents the benign baseline (approximately 0.1614).
Flooding attacks such as *ICMP flood*, *UDP flood*, and *TCP flood* exhibit clearly distinguishable `attack_prob` values, often well above 0.25, with *ICMP flood* peaking around 0.36. These high

values indicate strong model confidence and allow for reliable detection.
In contrast, attacks like *Slowloris*, *Aggressive Scan*, *Portscan*, and *OS Fingerprinting* present significantly lower `attack_prob` values, frequently falling below or very close to the benign average. This suggests that these attacks are more likely to be misclassified or overlooked, highlighting a potential weakness in the current detection model.
These results align with real-world test observations, where slower and stealthier attacks blend more seamlessly with normal traffic patterns, making them more challenging to detect. Future work should explore enhanced feature extraction or anomaly-based methods to improve classification confidence for these low-profile attacks.

# 9 Discussion

# 10 Conclusion/Future Work

einstellen das der pi immer eine gewissen bereich für die detection nutzten kann bei starken flooding angriffen, ist er nämlcih so stark eingeschränkt das er nicht einmal mehr feedback der event daten geben kann.

# Literaturverzeichnis


[20125] Prometheus Authors 2014-2025. *Prometheus - Open source metrics and monitoring for your systems and services*. Zugriff am 22.07.2025. 2025. URL: https://prometheus.io/.

[ACEA23] European Automobile Manufacturers Association (ACEA). *Interactive Map: Correlation Between Electric Car Sales and Charging Point Availability - 2023 Data*. https://www.acea.auto/figure/interactive-map-correlation-between-electric-car-sales-and-charging-point-availability-2023-data/. Accessed: 2025-07-09. 2023.

[AI-Definition] Iqbal H. Sarker. "AI-Based Modeling: Techniques, Applications and Research Issues Towards Automation, Intelligent and Smart Systems". In: *SN Computer Science* 3.2 (2022). Accessed: 2025-07-10, p. 158. DOI: 10.1007/s42979-022-01043-x. URL: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC8830986/.

[Ali20] Manoj Basnet; Mohd. Hasan Ali. "Deep Learning-based Intrusion Detection System for Electric Vehicle Charging Station". In: (2020). Accessed: 2025-07-16. URL: https://ieeexplore.ieee.org/document/9243152.

[Arma] Arm Developer. *Common event numbers*. URL: https://developer.arm.com/documentation/ddi0406/c/Debug-Architecture/The-Performance-Monitors-Extension/Event-numbers-and-mnemonics/Common-event-numbers?lang=en (visited on 07/24/2025).

[Armb] Arm Developer. *PMU events*. URL: https://developer.arm.com/documentation/100442/0100/debug-descriptions/pmu/pmu-events?lang=en (visited on 07/24/2025).

[Aro+21] Raman Arora et al. "Dropout: Explicit Forms and Capacity Control". In: (2021). Accessed: 2025-07-27. URL: https://proceedings.mlr.press/v139/arora21a/arora21a.pdf.

[ASVIN] ASVIN. *ASVIN*. https://asvin.io/. Accessed: 2025-07-10. 2025.

[BAN09] ARINDAM BANERJEE. "Anomaly Detection: A Survey". In: *University of Minnesota* (2009). Accessed: 2025-07-15. URL: http://cucis.ece.northwestern.edu/projects/DMS/publications/AnomalyDetection.pdf.

[BMM24] Pragathi B.C., Hrithik Maddirala, and Sneha M. “Implementing an Effective Infrastructure Monitoring Solution with Prometheus and Grafana”. In: (2024). Accessed: 2025-07-22. URL: https://www.ijcaonline.org/archives/volume186/number38/pragathi-2024-ijca-923873.pdf.

[Bre01] Leo Breiman. “Random Forests”. In: (2001). Accessed: 2025-07-15. URL: https://www.stat.berkeley.edu/~breiman/randomforest2001.pdf.

[ChargeIQ] ChargeIQ GmbH. *ChargeIQ GmbH*. https://chargeiq.de/. Accessed: 2025-07-10. 2025.

[Che+23] Chin-Ling Chen et al. “An Experimental Detection of Distributed Denial of Service Attack in CDX 3 Platform Based on Snort”. In: (2023). Accessed: 2025-07-24. URL: https://pmc.ncbi.nlm.nih.gov/articles/PMC10346265/.

[con25] hpc-wiki contributors. *Perf – Linux Performance Monitoring Tool*. https://hpc-wiki.info/hpc/Perf. Accessed: 2025-07-18. 2025.

[Coo+17] Tim Cooijmans et al. “RECURRENT BATCH NORMALIZATION”. In: (2017). Accessed: 2025-07-27. URL: https://arxiv.org/pdf/1603.09025.

[DecisionTreeExplain24] Bahzadour Taha Jijo and Adnan Mohsin Abdulazeez. “Decision Tree Explaination”. In: *Classification Based on Decision Tree Algorithm for Machine Learning* (2021). Accessed: 2025-07-09. URL: https://jastt.org/index.php/jasttpath/article/view/65/24.

[EDBF24] Sajjad Dadkhah Emmanuel Dana Buedi Ali A. Ghorbani and Raphael Lionel Ferreira. “Enhancing EV Charging Station Security Using a Multi-dimensional Dataset: CICEVSE2024”. In: (2024). Accessed: 2025-07-16. URL: https://link.springer.com/chapter/10.1007/978-3-031-65172-4_11.

[EntryPoints25] Md. Jakir Hossen V. Thiruppathy Kesavan. “Anomaly detection with grid sentinel framework for electric vehicle charging stations in a smart grid environment”. In: *Scientific Reports* 15.1 (2025). Accessed: 2025-07-09, p. 400. DOI: 10.1038/s41598-025-00400-z. URL: https://www.nature.com/articles/s41598-025-00400-z.

[EU2050] European Commission. *2050 long-term strategy: Climate-neutral Europe by 2050*. https://climate.ec.europa.eu/eu-action/climate-strategies-targets/2050-long-term-strategy_en. Accessed: 2025-07-09. 2020.

[EVSE-Cyber25] Paolo Bertoldi et al. “Cybersecurity for Electric Vehicle Charging Infrastructure: Challenges and Best Practices”. In: *World Electric Vehicle Journal* 15.12 (2025). Accessed: 2025-07-09, p. 571. DOI: 10.3390/wevj15120571. URL: https://www.mdpi.com/2032-6653/15/12/571.

[EVSE24] Sagar Babu Mitikiri et al. “Regression Based Anomaly Detection in Electric Vehicle State of Charge Fluctuations Through Analysis of EVCI Data”. In: *arXiv preprint arXiv:2401.01580v1* (2024). Accessed: 2025-07-09. URL: https://arxiv.org/abs/2401.01580v1.

[GDPR5] European Union. *General Data Protection Regulation (GDPR) – Article 5: Principles relating to processing of personal data*. https://gdpr-info.eu/art-5-gdpr/. Accessed: 2025-07-10. 2018.

[GDPR6] European Union. *General Data Protection Regulation (GDPR) – Article 6: Lawfulness of processing*. https://gdpr-info.eu/art-6-gdpr/. Accessed: 2025-07-10. 2018.

[Gee22a] GeeksforGeeks. *Bar plot in Matplotlib*. Accessed: 2025-07-23. 2022. URL: https://www.geeksforgeeks.org/bar-plot-in-matplotlib/.

[Gee22b] GeeksforGeeks. *Heatmap in Python*. Accessed: 2025-07-23. 2022. URL: https://www.geeksforgeeks.org/heatmap-in-python/.

[Gee22c] GeeksforGeeks. *Histogram using Matplotlib in Python*. Accessed: 2025-07-23. 2022. URL: https://www.geeksforgeeks.org/histogram-using-matplotlib-in-python/.

[Gee22d] GeeksforGeeks. *Line plot in Matplotlib - Python*. Accessed: 2025-07-23. 2022. URL: https://www.geeksforgeeks.org/line-plot-in-matplotlib/.

[Gee22e] GeeksforGeeks. *Scatter plot in Python using Matplotlib*. Accessed: 2025-07-23. 2022. URL: https://www.geeksforgeeks.org/scatter-plot-in-python-using-matplotlib/.

[Goo] *Exploratory Data Analysis for Feature Selection in Machine Learning*. https://services.google.com/fh/files/misc/exploratory_data_analysis_for_feature_selection_in_machine_learning.pdf. Accessed: 2025-07-23. Google LLC, 2024.

[HA04] V. J. Hodge and J. Austin. “A Survey of Outlier Detection Methodologies”. In: *Artificial Intelligence Review* (2004). Accessed: 2025-07-15. URL: https://www-users.york.ac.uk/~vjh5/myPapers/Hodge+Austin_OutlierDetection_AIRE381.pdf.

[Han25] Yannic Hanel. *Interior view of a wallbox*. Own image. 2025.

[Hod19] Daniel Hodges. *perf_exporter: A perf exporter for prometheus*. https://github.com/hodgesds/perf_exporter. Version v0.0.1, Zugriff am 18.07.2025. 2019.

[IEA22] International Energy Agency. *Global EV Outlook 2022*. Accessed: 2025-07-09. International Energy Agency, 2022. URL: https://www.iea.org/reports/global-ev-outlook-2022.

[KB15] Diederik P. Kingma and Jimmy Lei Ba. "ADAM: A METHOD FOR STOCHASTIC OPTIMIZATION". In: (2015). Accessed: 2025-07-27. URL: https://arxiv.org/pdf/1412.6980.

[Kot24] Shahir Kottilingal. "COMPARATIVE ANALYSIS OF FEATURE SELECTION TECHNIQUES FOR LSTM BASED NETWORK INTRUSION DETECTION MODELS". In: (2024). Accessed: 2025-07-26. URL: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4979065.

[Laux24] Lea Laux. "Übersicht eines Penetration Testings von Ladeinfrastruktur mit Schwerpunkt Ladestationen (im ReSiLENT-Projekt)". Summer semester 2024, submission: August 2024. Bachelorarbeit. Ostbayerische Technische Hochschule Regensburg, 2024.

[Lim+14] Robert V. Lim et al. "Computationally Efficient Multiplexing of Events on Hardware Counters". In: (2014). Accessed: 2025-07-18. URL: https://www.kernel.org/doc/ols/2014/ols2014-lim.pdf.

[LTZ08] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. "Isolation Forest". In: (2008). Accessed: 2025-07-15. URL: http://www.lamda.nju.edu.cn/publication/icdm08b.pdf.

[Mah22] Kiran Maharana. "A review: Data pre-processing and data augmentation techniques". In: *ScienceDirect - Knowledge and Information Systems* (2022). Accessed: 2025-07-15. URL: https://www.sciencedirect.com/science/article/pii/S2666285X22000565.

[Per] *Enum Hardware*. URL: https://docs.rs/perf-event/latest/perf_event/events/enum.Hardware.html (visited on 07/24/2025).

[per] perf wiki. *perf tutorial*. URL: https://perfwiki.github.io/main/tutorial/ (visited on 07/24/2025).

[Per19] PeruriVenkataAnusha. "Anomaly Detection Techniques". In: *International Journal of Innovative Technology and Exploring Engineering* (2019). Accessed: 2025-07-15. URL: https://www.ijitee.org/wp-content/uploads/papers/v9i1/A3910119119.pdf.

[Rah+25] Md Rayhanur Rahman et al. “Mining Temporal Attack Patterns from Cyberthreat Intelligence Reports”. In: (2025). Accessed: 2025-07-27. URL: https://ar5iv.labs.arxiv.org/html/2401.01883.

[Ram+18] Daniel Ramos et al. “Deconstructing Cross-Entropy for Probabilistic Binary Classifiers”. In: (2018). Accessed: 2025-07-27. URL: https://www.mdpi.com/1099-4300/20/3/208.

[ReSiLENT] ReSiLENT Project. *ReSiLENT Project*. https://resilent-project.de/. Accessed: 2025-07-10. 2025.

[Sae] Mehreen Saeed. *A Gentle Introduction To Sigmoid Function*. URL: https://machinelearningmastery.com/a-gentle-introduction-to-sigmoid-function/ (visited on 07/27/2025).

[SM19] Ralf C. Staudemeyer and Eric Rothstein Morris. “Understanding LSTM – a tutorial into Long Short-Term Memory Recurrent Neural Networks”. In: (2019). Accessed: 2025-07-09. URL: https://arxiv.org/pdf/1909.09586.

[SM24] Inna Stetsenko and Anton Myroniuk. “SOFTWARE FOR COLLECTING AND ANALYZING METRICS IN HIGHLY LOADED APPLICATIONS BASED ON THE PROMETHEUS MONITORING SYSTEM”. In: (2024). Accessed: 2025-07-22. URL: https://ela.kpi.ua/server/api/core/bitstreams/8138e408-804b-45f6-9842-9e7b41e50dc0/content.

[StoehrMobility] Stöhr Mobility GmbH. *Stöhr Mobility GmbH*. https://www.stoehr-mobility.de/. Accessed: 2025-07-10. 2025.

[The] The Linux Kernel Archives. *Kernel Trace Events – kmem*. URL: https://www.kernel.org/doc/html/v5.14/trace/events-kmem.html (visited on 07/24/2025).

[The25] The pandas development team. *pandas documentation, version 2.3.1*. Accessed: 2025-07-23. 2025. URL: https://pandas.pydata.org/docs/.

[Wea15] Vincent M. Weaver. “Linux perf event Features and Overhead”. In: (2015). Accessed: 2025-07-18, pp. 1–8.

[Wt25] Michael Waskom and the Seaborn development team. *seaborn: statistical data visualization*. Accessed: 2025-07-23. 2025. URL: https://seaborn.pydata.org/.

# Anhang

```go
package main
import (
    "fmt"
    "io/ioutil"
    "net/http"
    "os/exec"
    "strconv"
    "strings"
    "time"

    "gopkg.in/yaml.v2"
)

// Config represents the structure of the YAML configuration file.
// It includes high-level performance counters (HCP), software events (SW), and hardware events (HW).
type Config struct {
    HCP []string `yaml:"hcp"`
    SW  []string `yaml:"sw"`
    HW  []string `yaml:"hw"`
}

// Global variable to store the most recent performance data.
var perfData string

// getPerfData runs `perf stat` with the specified events for a short duration (0.1s)
// and returns the raw output as a string.
func getPerfData(events []string) (string, error) {
    eventList := strings.Join(events, ",")
    cmd := exec.Command("perf", "stat", "-e", eventList, "--", "sleep", "0.1")
    output, err := cmd.CombinedOutput()
    if err != nil {
        return "", err
    }
    return string(output), nil
}

// parsePerfData filters and extracts only the relevant lines from perf output,
// based on the provided list of events.
func parsePerfData(data string, events []string) string {
    lines := strings.Split(data, "\n")
    var result []string
    for _, line := range lines {
```

```
43          for _, event := range events {
44              if strings.Contains(line, event) {
45                  result = append(result, line)
46                  break
47              }
48          }
49      }
50      return strings.Join(result, "\n")
51  }
52
53  // formatForPrometheus converts the filtered performance data into
        Prometheus-compatible metrics format.
54  func formatForPrometheus(data string) string {
55      lines := strings.Split(data, "\n")
56      var result []string
57      for _, line := range lines {
58          parts := strings.Fields(line)
59          if len(parts) >= 2 {
60              // Clean up the metric name
61              metricName := strings.ReplaceAll(parts[1], ".", "_")
62              // metricName = strings.ReplaceAll(metricName, "-", "_")
63              // Remove decimal points from the number and parse it
64              metricValue := strings.ReplaceAll(parts[0], ".", "")
65              if value, err := strconv.ParseFloat(metricValue, 64); err == nil
                    {
66                  // Format value appropriately for Prometheus (integer or
                        float)
67                  if value == float64(int(value)) {
68                      result = append(result, fmt.Sprintf("%s %d", metricName,
                            int(value)))
69                  } else {
70                      result = append(result, fmt.Sprintf("%s %f", metricName,
                            value))
71                  }
72              }
73          }
74      }
75      return strings.Join(result, "\n")
76  }
77
78  // metricsHandler handles HTTP requests and returns the formatted
        performance metrics.
79  func metricsHandler(w http.ResponseWriter, r *http.Request) {
80      fmt.Fprintf(w, formatForPrometheus(perfData))
81  }
82
83  // chunkEvents splits the list of events into smaller chunks of a given
        size.
```

```
84 // This helps avoid issues with perf's limit on the number of
       simultaneous counters.
85 func chunkEvents(events []string, chunkSize int) [][]string {
86     var chunks [][]string
87     for i := 0; i < len(events); i += chunkSize {
88         end := i + chunkSize
89         if end > len(events) {
90             end = len(events)
91         }
92         chunks = append(chunks, events[i:end])
93     }
94     return chunks
95 }
96
97 // validateEvent checks if a given performance event is available on the
       system by using 'perf list'.
98 func validateEvent(event string) bool {
99     cmd := exec.Command("perf", "list")
100    output, err := cmd.CombinedOutput()
101    if err != nil {
102        fmt.Println("Error listing perf events:", err)
103        return false
104    }
105    return strings.Contains(string(output), event)
106 }
107
108 func main() {
109    // Load the YAML configuration file
110    config := Config{}
111    data, err := ioutil.ReadFile("config.yml")
112    if err != nil {
113        fmt.Println("Error reading config file:", err)
114        return
115    }
116    err = yaml.Unmarshal(data, &config)
117    if err != nil {
118        fmt.Println("Error parsing config file:", err)
119        return
120    }
121
122    // Validate and collect all available events from the config
123    var validEvents []string
124    for _, event := range append(config.HCP, config.SW...) {
125        if validateEvent(event) {
126            validEvents = append(validEvents, event)
127        }
128    }
129    for _, event := range config.HW {
```

```
130         if validateEvent(event) {
131             validEvents = append(validEvents, event)
132         }
133     }
134 
135     // Split the events into manageable chunks (TODO: dynamically adjust
        based on CPU performance)
136     eventChunks := chunkEvents(validEvents, 6)
137 
138     // Start a background goroutine to periodically collect performance
        data
139     go func() {
140         for {
141             var allPerfData []string
142             for _, chunk := range eventChunks {
143                 data, err := getPerfData(chunk)
144                 if err != nil {
145                     fmt.Println("Error:", err)
146                     allPerfData = append(allPerfData, "Error fetching data")
147                 } else {
148                     allPerfData = append(allPerfData, parsePerfData(data,
                        chunk))
149                 }
150             }
151             // Combine data from all chunks into one string for Prometheus
                export
152             perfData = strings.Join(allPerfData, "\n")
153             time.Sleep(100 * time.Millisecond)
154         }
155     }()
156 
157     // Start the HTTP server that exposes the `/metrics` endpoint for
        Prometheus scraping
158     http.HandleFunc("/metrics", metricsHandler)
159     fmt.Println("Starting server on :9011")
160     if err := http.ListenAndServe(":9011", nil); err != nil {
161         fmt.Println("Error starting server:", err)
162     }
163 }
```

Listing 16: Go script for performance monitoring via perf for Prometheus

```
1 import requests
2 import csv
3 from datetime import datetime, timedelta
4 from collections import defaultdict
5 import numpy as np
6 
7 from config_allevents import FEATURES
```

```

PROMETHEUS_URL = "http://192.168.1.25:9090"
STEP = "10"  # seconds
CHUNK_HOURS = 2

start_time = datetime.fromisoformat("2025-07-22T18:00:00.000000")
end_time = datetime.fromisoformat("2025-07-23T14:40:00.000000")

def build_chunks(start, end, chunk_hours):
    chunks = []
    chunk_delta = timedelta(hours=chunk_hours)
    current_start = start
    while current_start < end:
        current_end = min(current_start + chunk_delta, end)
        chunks.append((
            current_start.isoformat("T") + "Z",
            current_end.isoformat("T") + "Z"
        ))
        current_start = current_end
    return chunks

def query_metric_chunked(metric, chunks):
    all_results = []
    for chunk_start, chunk_end in chunks:
        url = f"{PROMETHEUS_URL}/api/v1/query_range"
        params = {
            "query": metric,
            "start": chunk_start,
            "end": chunk_end,
            "step": STEP,
        }
        response = requests.get(url, params=params)
        response.raise_for_status()
        all_results.extend(response.json()["data"]["result"])
    return all_results

def main():
    chunks = build_chunks(start_time, end_time, CHUNK_HOURS)
    data = defaultdict(dict)
    timestamps = set()
    values_dict = {}
    metric_query_map = {}

    for feat in FEATURES:
        prom_name = feat.replace("-", "_")
        url = f"{PROMETHEUS_URL}/api/v1/query"
        hit = False

```

```
56          # 1. sum(prom_name) abfragen
57          query = f"sum({prom_name})"
58          params = {'query': query}
59          try:
60              resp = requests.get(url, params=params)
61              resp.raise_for_status()
62              results = resp.json()['data']['result']
63              if results:
64                  value = float(results[0]['value'][1])
65                  values_dict[feat] = value
66                  metric_query_map[feat] = query
67                  hit = True
68          except Exception:
69              pass
70
71          if hit:
72              continue
73
74          # 2. sum(perf_prom_name) abfragen
75          prom_name_perf = 'perf_' + prom_name
76          query = f"sum({prom_name_perf})"
77          params = {'query': query}
78          try:
79              resp = requests.get(url, params=params)
80              resp.raise_for_status()
81              results = resp.json()['data']['result']
82              if results:
83                  value = float(results[0]['value'][1])
84                  values_dict[feat] = value
85                  metric_query_map[feat] = query
86                  hit = True
87          except Exception:
88              pass
89
90      values_array = []
91      feature_names_for_array = []
92
93      for feat in FEATURES:
94          feature_names_for_array.append(feat)
95          values_array.append(values_dict.get(feat, np.nan))
96
97      for feat, querystr in metric_query_map.items():
98          results = query_metric_chunked(querystr, chunks)
99          for series in results:
100             for timestamp, value in series["values"]:
101                 ts = datetime.utcfromtimestamp(float(timestamp)).
                        isoformat()
102                 data[ts][feat] = value
```

```
                    timestamps.add(ts)

    sorted_timestamps = sorted(timestamps)

    all_columns = list(FEATURES)

    with open("evse_metrics_wide.csv", "w", newline="") as f:
        writer = csv.writer(f)
        header = ["timestamp"] + all_columns
        writer.writerow(header)
        for ts in sorted_timestamps:
            row = [ts] + [data[ts].get(col, "") for col in all_columns]
            writer.writerow(row)

if __name__ == "__main__":
    main()
```

Listing 17: Python script for scraping performance monitoring from Prometheus into an csv file

```
import subprocess
import time
from datetime import datetime

commands = [
    ("Slowloris", ["python3", "slowloris.py", "192.168.1.25", "-s", "500"]),
    ("UDP Flood", ["sudo", "hping3", "--flood", "--udp", "-p", "80", "192.168.1.25"]),
    ("ICMP Flood", ["sudo", "hping3", "--flood", "-1", "192.168.1.25"]),
    ("TCP Flood", ["sudo", "hping3", "--flood", "-p", "80", "192.168.1.25"]),
    ("Synonymous Flood", ["sudo", "hping3", "--flood", "--rand-source", "-p", "80", "192.168.1.25"]),
    ("Portscan", ["bash", "-c", "for i in {1..1000}; do sudo nmap -sT 192.168.1.25; sleep 1; done"]),
    ("OS Fingerprinting", ["bash", "-c", "for i in {1..1000}; do sudo nmap -O 192.168.1.25; sleep 1; done"]),
    ("Aggressive Scan", ["bash", "-c", "for i in {1..1000}; do sudo nmap -A 192.168.1.25; sleep 1; done"]),
]

LOG_FILE = "timestamp_attacks.txt"
RUN_TIME = 15 * 60      # 15 Minuten
BREAK_TIME = 20 * 60    # 20 Minuten

def log_msg(msg):
    with open(LOG_FILE, "a") as f:
        f.write(msg + "\n")
```

```
23
24 while True:
25     for name, cmd in commands:
26         start_time = datetime.now()
27         print(f"Starte {name}: {' '.join(cmd)}")
28         log_msg(f"Start: {start_time.strftime('%Y-%m-%d %H:%M:%S')} -
               Angriff: {name} - Befehl: {' '.join(cmd)}")
29         try:
30             proc = subprocess.Popen(cmd)
31             proc.wait(timeout=RUN_TIME)
32             note = "Erfolgreich beendet"
33         except subprocess.TimeoutExpired:
34             proc.terminate()
35             try:
36                 proc.wait(10)
37             except Exception:
38                 proc.kill()
39             note = "Nach 15 Minuten abgebrochen"
40         end_time = datetime.now()
41         log_msg(f"Ende: {end_time.strftime('%Y-%m-%d %H:%M:%S')} ({note})
               ")
42         log_msg("Pause startet jetzt.\n")
43         time.sleep(BREAK_TIME)
```

Listing 18: Python script for automatet attacks and storage of the timestamps

content des beigefügten Datenträgers:

- ...
- ...

# A Domändenmodell

Ein toller Anhang, der nicht nur als „*Müllhalde*“ genutzt wird, sondern in dem Bilder und contente auch mit eigenen Worten erklärt werden und den man auch für sich alleine lesen kann. Es sollten auch Referenzen auf die zugehörige ausführliche Behandlung im Hauptteil inklusive Seitenangabe mit `\pageref` gegeben werden.

## Screenshot

Unterkategorie, die nicht im contentsverzeichnis auftaucht.